

Sayısal İşaret İşleme

Modülasyon, Demodülasyon ve IQ İşleme

C++ ile Sıfırdan: DFT/FFT, Filtreler, AM/FM, PSK/QAM ve Tam Bir SDR Alıcısı

Bu belge önce **örnekleme**, **Nyquist** ve **karmaşık sayılar** gibi temelleri; sonra **DFT/FFT ile frekans analizini**; ardından **FIR/IIR filtreleri**; sonra işin kalbi olan **IQ (karmaşık bant temeli)** kavramını; en sonda ise **AM, FM, PM ve sayısal modülasyonların** tam modem örneklerini anlatır. Kapanışta gerçek bir **FM radyo alıcısının** (RTL-SDR IQ akışından sese) uçtan uca hattı yazılır. Tüm kod örnekleri İngilizce isimlendirmeye, standart C++17 ile, harici kütüphaneye ihtiyaç duymadan derlenecek şekilde yazılmıştır.

Kapsam — Kısım 1 (Temeller): işaret nedir • analog vs sayısal • örnekleme & Nyquist • aliasing • kuantalama & ADC • karmaşık sayılar & fazörler • sinyal üretimi & WAV G/Ç.

Kapsam — Kısım 2 (Frekans Analizi): Fourier fikri • DFT • FFT (radix-2, özyinelemeli & yinelemeli) • pencereleme • genlik/faz spektrumu • güç spektral yoğunluğu.

Kapsam — Kısım 3 (Filtreler): konvolüsyon • FIR & doğrusal faz • pencere yöntemiyle tasarım • IIR & biquad • kutup-sıfır • hareketli ortalama & CIC.

Kapsam — Kısım 4 (IQ & Bant Temeli): analitik işaret & Hilbert • I/Q anlamı • NCO & mikser ile frekans kaydırma • desimasyon/interpolasyon • takım yıldızı (constellation).

Kapsam — Kısım 5 (Modülasyon): AM (zarf & senkron) • FM (IQ diskriminatör) • PM • ASK/FSK/PSK/QAM • QPSK modem • Costas & PLL • zamanlama kurtarma.

Kapsam — Kısım 6 (Uçtan Uca Alıcı): IQ dosya formatları • RTL-SDR akışı • tam WBFM radyo hattı • stereo & de-emphasis • performans notları.

Kapsam — Kısım 7 (İleri): stereo FM çözücü • AGC • uçtan uca QPSK modem • OFDM (IFFT/FFT) • konvolüsyonel kod & Viterbi.

Kapsam — Kısım 8 (Projeler): DTMF çözücü (Goertzel) • ses üzerinden FSK veri iletimi • terminal spektrum analizörü • enstrüman akort cihazı • CMake ile derleme.

Kapsam — Kısım 9 (Matematik): diklik & Euler • Fourier zinciri (FS/FT/DTFT/DFT) • FFT ispatı • örnekleme teoreminin türetimi • konvolüsyon & modülasyon teoremleri • Parseval • Bessel & Carson • analitik işaret & anlık frekans.

Hazırlanma Tarihi: 20 Temmuz 2026

C++17 • <complex> & <vector> • Nyquist–Shannon • RTL-SDR • CMake

Contents

1 İşaret Nedir? Analogdan Sayısala	2
1.1 Ayırık-Zamanlı İşaretin Gösterimi	2
2 Örnekleme, Nyquist ve Aliasing	3
2.1 Aliasing: Kimlik Değiştiren Frekanslar	3
2.2 Aliasing'i Sese Çevirmek	4
3 Kuantalama ve ADC	4
4 Karmaşık Sayılar ve Fazörler	5
4.1 C++'ta <code>std::complex</code> ile Çalışmak	5
5 Sinyal Üretimi ve WAV Dosyaları	6
6 Fourier Fikri	8
6.1 DFT'nin Doğrudan Uygulaması	8
7 FFT: Hızlı Fourier Dönüşümü	9
7.1 Özyinelemeli Radix-2 FFT	9
7.2 Yerinde (In-Place) Yinelemeli FFT	9
8 Pencereleme (Windowing)	11
8.1 Güç Spektral Yoğunluğu (PSD)	12
9 Konvolüsyon ve FIR Filtreler	13
9.1 Temel FIR Uygulaması	13
9.2 Pencere Yöntemiyle Alçak Geçiren FIR Tasarımı	14
10 IIR Filtreler ve Biquad	15
10.1 Hareketli Ortalama ve CIC: Ucuz Filtreler	16
11 IQ Nedir? Analitik İşaret	17
11.1 Analitik İşaret ve Hilbert Dönüşümü	17
12 NCO, Mikser ve Frekans Kaydırma	18
13 Desimasyon ve Örnekleme Hızı Dönüşümü	19
13.1 Karmaşık (IQ) Desimasyon	20
14 Genel Bakış: Modülasyon Neden Var?	21
15 AM: Genlik Modülasyonu	21
15.1 AM Modülatörü	22
15.2 Zarf (Envelope) Detektörü ile Demodülasyon	22
15.3 Senkron (Coherent) AM Demodülasyonu	23
16 FM: Frekans Modülasyonu	23
16.1 FM'in Matematik: Modülasyon İndeksi, Bessel ve Carson	23
16.2 FM Modülatörü	24
16.3 FM Demodülasyonu: IQ Diskriminatör	25
16.4 De-emphasis: FM'in Son Rötüşü	26
17 PM: Faz Modülasyonu	26

18 Sayısal Modülasyonlar: ASK, FSK, PSK, QAM	27
18.1 BPSK ve QPSK: Faz Kaydırmalı Anahtarlama	27
18.2 FSK: Frekans Kaydırmalı Anahtarlama	28
18.3 QAM: Genlik + Faz Birlikte	29
19 Darbe Şekillendirme (Pulse Shaping)	29
20 Taşıyıcı ve Zamanlama Kurtarma: PLL, Costas, Timing	30
20.1 Faz Kilitlemeli Döngü (PLL)	30
20.2 Costas Döngüsü: PSK için Taşıyıcı Kurtarma	31
20.3 Sembol Zamanlama Kurtarma (Gardner)	32
21 IQ Dosya Formatları ve RTL-SDR	33
21.1 RTL-SDR Baytlarını Karmaşık Bant Temeline Çevirme	33
22 Tam WBFM Alıcı Hattı	34
23 Performans ve Sonraki Adımlar	35
23.1 Optimizasyon Notları	35
23.2 Üretimde Kullanılan Kütüphaneler	36
23.3 Nereye Bakmalı	36
24 Stereo FM Çözücü (Tam Uygulama)	37
25 Otomatik Kazanç Kontrolü (AGC)	38
26 Uçtan Uca Sayısal Modem: QPSK Vericiden Alıcıya	39
26.1 Verici (Modülatör)	39
26.2 Kanal Modeli	40
26.3 Alıcı (Demodülatör)	40
27 OFDM: Çok Taşıyıcılı Modülasyon	41
28 Kanal Kodlaması: Konvolüsyonel Kod ve Viterbi	42
28.1 Konvolüsyonel Kodlayıcı	42
28.2 Viterbi Çözücü	43
29 Proje 1 — DTMF (Telefon Tuşu) Çözücü	45
30 Proje 2 — Ses Üzerinden Veri İletimi (FSK Modem)	46
31 Proje 3 — Terminal Spektrum Analizörü	48
32 Proje 4 — Enstrüman Akort Cihazı (Pitch Detection)	49
33 Projeleri Derleme (CMake)	50
34 Karmaşık Üsteller ve Diklik (Orthogonality)	52
34.1 Diklik İlişkisi	52
35 Fourier Zinciri: Seriden DFT'ye	52
35.1 Fourier Serisi ve Dönüşümü	53
35.2 DTFT'den DFT'ye: Kodun Türetimi	53
35.3 FFT'nin Böl-ve-Fethet İspatı	53

36 Örneklemeye Teoreminin Türetimi	54
37 Konvolüsyon ve Modülasyon Teoremleri	54
37.1 Konvolüsyon Teoremi	54
37.2 Modülasyon (Frekans Kaydırma) Teoremi	54
37.3 Parseval Teoremi	55
38 Analitik İşaret ve Anlık Frekans	55
38.1 Anlık Frekansın Tanımı	55
38.2 Polar Diskriminatörün Türetimi	55
38.3 Hilbert Dönüşümü ve Analitik İşaretin Türetimi	55

KISIM 1

İşaret İşlemenin Temelleri

1. İşaret Nedir? Analogdan Sayısala

Bir **işaret** (signal), bilgi taşıyan ve zamana (veya uzaya) göre değişen bir büyüklüktür. Bir mikrofonun zara çarpan hava basıncını gerilime çevirmesi, bir antenin havadaki elektromanyetik alanı akıma çevirmesi, bir sıcaklık sensörünün ısıyı volta çevirmesi — hepsi işaret üretir. İşaret işleme, bu büyüklükleri *ölçmek, dönüştürmek, filtrelemek, analiz etmek ve yeniden üretmek* sanatıdır.

İşaretleri iki eksende sınıflandırırız:

	Zaman eksen	Genlik eksen
Analog	sürekli ($t \in \mathbb{R}$)	sürekli (her değeri alabilir)
Ayrık-zamanlı	kesikli ($n \in \mathbb{Z}$)	sürekli
Sayısal (dijital)	kesikli	kesikli (kuantalanmış)

Gerçek dünya analogdur; bilgisayarlar ise sayısaldır. Aradaki köprü iki bileşendir: **ADC** (Analog-to-Digital Converter) analog işareti belirli aralıklarla ölçüp sayıya çevirir; **DAC** (Digital-to-Analog Converter) ise sayı dizisini tekrar sürekli bir gerilime çevirir.

Bilgi

Sayısal işaret işlemenin (DSP) tüm gücü şu basit gerçekten gelir: Bir işareti sayı dizisine çevirdiğimizde, onun üzerinde *aritmetik* yapabiliriz. Filtreleme bir çarpma-toplama, modülasyon bir çarpma, frekans analizi bir matris dönüşümü hâline gelir. Analogda pahalı devrelerle yapılan her şey, sayısalda birkaç satır koda dönüşür.

1.1 Ayrık-Zamanlı İşaretin Gösterimi

Ayrık bir işareti $x[n]$ ile gösteririz; burada n tamsayı örnek indeksidir. Eğer örnekler T_s saniye aralıkla alınıyorsa, **örnekleme frekansı** $f_s = 1/T_s$ 'dir ve n 'inci örnek gerçek zamanda $t = n T_s = n/f_s$ anına karşılık gelir. C++'ta bir işaret çoğunlukla bir dizidir:

```

1 #include <vector>
2 #include <cmath>
3
4 // Gerçek-değerli bir işaret genelde float/double vektörüdür.
5 using Signal = std::vector<float>;

```

```

6
7 // fs Hz örnekleme hızıyla, f Hz'lik saf bir sinüs üret.
8 // x[n] = A * sin(2*pi*f*n/fs + phi)
9 Signal makeSine(double freqHz, double fs, double durationSec,
10                 double amplitude = 1.0, double phaseRad = 0.0) {
11     const size_t N = static_cast<size_t>(durationSec * fs);
12     Signal x(N);
13     const double w = 2.0 * M_PI * freqHz / fs; // örnek başına faz artışı (rad)
14     for (size_t n = 0; n < N; ++n)
15         x[n] = static_cast<float>(amplitude * std::sin(w * n + phaseRad));
16     return x;
17 }

```

Not

Buradaki $w = 2\pi f/f_s$ niceliğine *sayısal açısal frekans* denir ve birimi “radyan/örnek”tir. DSP’de neredeyse her formül f_s ’e göre normalize edilmiş bu w cinsinden yazılır; çünkü algoritma açısından önemli olan mutlak frekans değil, örnekleme hızına *oranıdır*.

2. Örnekleme, Nyquist ve Aliasing

DSP’nin en temel teoremi **Nyquist–Shannon örnekleme teoremidir**: bant genişliği B olan (yani en yüksek frekans bileşeni B Hz olan) bir işareti kayıpsız temsil etmek için örnekleme frekansı en az $2B$ olmalıdır:

$$f_s > 2f_{\max} \quad (\text{Nyquist kriteri})$$

$f_s/2$ değerine **Nyquist frekansı** denir ve sayısal dünyada temsil edebileceğimiz en yüksek frekanstır.

2.1 Aliasing: Kimlik Değiştiren Frekanslar

Nyquist kriterini ihlal edersek — yani $f_{\max}$ ’tan daha düşük bir hızla örnekleme yaparsak — yüksek frekanslar düşük frekanslar gibi *kılık değiştirir*. Buna **aliasing** (örtüşme) denir. Matematiksel olarak, f ve $f + kf_s$ frekansları (herhangi bir tamsayı k) örneklendiğinde *tamamen aynı* örnek dizisini üretir:

$$\sin(2\pi(f + kf_s)n/f_s) = \sin(2\pi f n/f_s + 2\pi kn) = \sin(2\pi f n/f_s)$$

çünkü $2\pi kn$ terimi tam bir tur katıdır. Aşağıdaki kod bunu deneysel olarak gösterir: $f_s = 1000$ Hz’te örneklen 300 Hz ve 700 Hz’lik sinüsler bit-bit aynı örnekleri verir ($700 = 1000 - 300$, yani 700 Hz, -300 Hz’in aliasıdır).

```

1 #include <stdio>
2 #include <cmath>
3
4 int main() {
5     const double fs = 1000.0; // örnekleme frekansı
6     const double f1 = 300.0, f2 = 700.0;
7     printf(" n |   300 Hz   |   700 Hz   | fark\n");
8     for (int n = 0; n < 8; ++n) {
9         double a = std::sin(2*M_PI*f1*n/fs);
10        double b = std::sin(2*M_PI*f2*n/fs);
11        printf("%2d | %+.6f | %+.6f | %.1e\n", n, a, b, std::fabs(a-b));
12    }
13    // fark ~ 1e-16: iki farklı frekans, ayırt edilemez örnekler üretir!
14    return 0;
15 }

```

Dikkat

Aliasing geri döndürülemez bir bilgi kaybıdır. Örnekleme yapıldıktan *sonra* hangi frekansın “gerçek” olduğunu bilmenin yolu yoktur. Bu yüzden her gerçek ADC’nin önünde, Nyquist üstü frekansları örneklemeden *önce* kesen bir analog **anti-aliasing (kıvrım önleyici) filtre** bulunur.

2.2 Aliasing’i Sese Çevirmek

Aliasing’i somutlaştırmak için, $f_s/2$ ’yi aşan bir frekansın nereye katlandığını hesaplayan küçük bir yardımcı yazalım. Katlanan frekans $f_{\text{alias}} = |f - kf_s|$ olup, sonucu $[0, f_s/2]$ aralığına indirir:

```

1 #include <cmath>
2
3 // Bir f frekansının fs ile örneklendiğinde nereye "alias" olacağını verir.
4 double aliasedFrequency(double f, double fs) {
5     double folded = std::fmod(f, fs); // [0, fs) aralığına indir
6     if (folded < 0) folded += fs;
7     if (folded > fs / 2.0) folded = fs - folded; // Nyquist'in üstünü katla
8     return folded;
9 }
10
11 // Örnek: fs = 48 kHz
12 // aliasedFrequency(1000, 48000) -> 1000 (sorun yok)
13 // aliasedFrequency(47000, 48000) -> 1000 (47 kHz, 1 kHz gibi duyulur!)
14 // aliasedFrequency(25000, 48000) -> 23000 (Nyquist'in hemen üstü katlanır)

```

3. Kuantalama ve ADC

Örnekleme zaman eksenini kesikli yapar; **kuantalama** ise genlik eksenini kesikli yapar. Bir B -bitlik ADC, giriş aralığını 2^B seviyeye böler. Örneğin 16-bit bir ADC, gerilim aralığını 65536 basamağa böler. Her örnek en yakın basamağa yuvarlanır; aradaki fark **kuantalama gürültüsüdür**.

Bir ideal kuantalayıcının sinyal-gürültü oranı (SNR) yaklaşık şudur:

$$\text{SNR}_{\text{dB}} \approx 6,02 B + 1,76$$

Yani her ek bit yaklaşık 6 dB dinamik aralık kazandırır. 16-bit ≈ 98 dB, 12-bit (tipik SDR) ≈ 74 dB.

```

1 #include <cmath>
2 #include <cstdint>
3
4 // [-1, 1] aralığındaki float'ı B-bit işaretli tamsayıya kuantala.
5 int32_t quantize(float x, int bits) {
6     const int32_t levels = (1 << (bits - 1)) - 1; // örn. 16-bit -> 32767
7     float clamped = std::fmax(-1.0f, std::fmin(1.0f, x));
8     return static_cast<int32_t>(std::lround(clamped * levels));
9 }
10
11 // Kuantalanmış değeri geri float'a çevir (DAC'ın yaptığı).
12 float dequantize(int32_t q, int bits) {
13     const int32_t levels = (1 << (bits - 1)) - 1;
14     return static_cast<float>(q) / levels;
15 }

```

İpucu

RTL-SDR gibi ucuz alıcılar 8-bit örnekler verir (her I ve Q için bir bayt, 0..255, orta nokta 127.5). Bu yüzden bir SDR programının ilk adımı neredeyse her zaman bu baytları $[-1, 1]$ aralığına ölçekleyip float'a çevirmektir. Bunun kodunu Kısım 6'da göreceğiz.

4. Karmaşık Sayılar ve Fazörler

Modülasyon ve IQ işlemenin dilini konuşabilmek için **karmaşık sayılar** şarttır. Bir karmaşık sayı $z = a + jb$ 'dir; burada $j = \sqrt{-1}$ (mühendisler i yerine j kullanır, çünkü i akımı temsil eder). Gerçek kısım $a = \Re\{z\}$, sanal kısım $b = \Im\{z\}$ 'dir.

En kritik ilişki **Euler formülüdür**:

$$e^{j\theta} = \cos \theta + j \sin \theta$$

Bu, birim çember üzerinde θ açısındaki bir noktadır. Zamanla dönen bir karmaşık üstel $e^{j2\pi ft}$ 'ye **fazör** denir ve f frekansında dönen bir vektör olarak düşünülür. İşte modülasyonun tüm geometrisi budur: bilgiyi bu dönen vektörün *genliğine*, *açısına* veya *dönme hızına* gömeriz.

Büyüklik**Anlamı ve C++ karşılığı**

Genlik (magnitude)	$ z = \sqrt{a^2 + b^2}$ — <code>std::abs(z)</code> . AM'de bilgi burada.
Faz (argument)	$\angle z = \text{atan2}(b, a)$ — <code>std::arg(z)</code> . PM/PSK'de bilgi burada.
Anlık frekans	fazın türevi $\frac{d}{dt}\angle z$. FM'de bilgi burada.
Eşlenik	$z^* = a - jb$ — <code>std::conj(z)</code> . Frekans kaydırmada kullanılır.
Güç	$ z ^2 = a^2 + b^2$ — <code>std::norm(z)</code> (dikkat: karekök <i>yok</i>).

Dikkat

C++ standart kütüphanesinde `std::norm(z)` matematiksel normu (uzunluğu) *değil*, uzunluğun *karesini* ($|z|^2$) döndürür. Uzunluk için `std::abs(z)` kullanın. Bu, DSP kodunda sık yapılan bir hatadır.

4.1 C++'ta `std::complex` ile Çalışmak

Standart `<complex>` başlığı bize hazır bir karmaşık aritmetik verir. SDR dünyasında yaygın olarak `std::complex<float>` kullanılır (bellek ve hız için); GNU Radio'daki `gr_complex` de tam olarak budur.

```

1 #include <complex>
2 #include <cstdio>
3
4 using cf32 = std::complex<float>; // I = real, Q = imag
5
6 int main() {
7     cf32 z(3.0f, 4.0f);           // 3 + 4j (I=3, Q=4)
8
9     printf("z      = %.1f + %.1fj\n", z.real(), z.imag());
10    printf("|z|    = %.4f\n", std::abs(z)); // 5.0 (genlik)
11    printf("|z|^2  = %.4f\n", std::norm(z)); // 25.0 (guc, karekoksuz!)

```



```

12 printf("faz      = %.4f rad\n", std::arg(z)); // atan2(4,3) = 0.9273
13 printf("eslenik  = %.1f + %.1fj\n",
14       std::conj(z).real(), std::conj(z).imag()); // 3 - 4j
15
16 // Euler: bir birim fazor uret (45 derece)
17 cf32 p = std::polar(1.0f, 3.14159265f / 4.0f); // e^(j*pi/4)
18 printf("polar    = %.4f + %.4fj\n", p.real(), p.imag()); // ~0.707 + 0.707j
19
20 // İki fazoru carpma aci'lerini TOPLAR (frekans kaydirmanin ozu)
21 cf32 a = std::polar(1.0f, 0.3f);
22 cf32 b = std::polar(1.0f, 0.5f);
23 printf("carpim aci = %.4f (=0.8?)\n", std::arg(a * b)); // 0.8
24 return 0;
25 }

```

Bilgi

Neden karmaşık sayılar? Çünkü **çarpım açıları toplar**: $e^{j\alpha} \cdot e^{j\beta} = e^{j(\alpha+\beta)}$. Bir işareti $e^{-j2\pi f_0 t/f_s}$ ile çarpma, onu frekans ekseninde $-f_0$ kadar kaydırır — işte tüm mikserlerin, kanal seçicilerin ve demodülatörlerin çalışma prensibi budur. Gerçek sinüslerle bunu yapmaya çalışsaydık, çarpım hem toplam hem fark frekansını üretir ve istenmeyen bileşeni filtrelememiz gerekirdi.

5. Sinyal Üretimi ve WAV Dosyaları

Sonuçlarımızı duyabilmek ve grafiklemek için sinyalleri diske yazmak gerekir. En yaygın format **WAV**'dir (RIFF konteyneri, PCM veri). Aşağıda harici kütüphane olmadan 16-bit mono/stereo WAV yazan tam bir yardımcı var — bu belge boyunca üretilen sesleri kaydetmek için kullanacağız.

```

1  #include <cstdint>
2  #include <cstdio>
3  #include <fstream>
4  #include <vector>
5  #include <cmath>
6
7  // 16-bit PCM WAV yazici. samples: [-1,1] araligindaki interleaved kanallar.
8  void writeWav(const std::string& path, const std::vector<float>& samples,
9              uint32_t sampleRate, uint16_t channels = 1) {
10     std::ofstream f(path, std::ios::binary);
11     const uint32_t nSamples = static_cast<uint32_t>(samples.size());
12     const uint16_t bitsPerSample = 16;
13     const uint16_t blockAlign = channels * bitsPerSample / 8;
14     const uint32_t byteRate = sampleRate * blockAlign;
15     const uint32_t dataBytes = nSamples * sizeof(int16_t);
16
17     auto put32 = [&](uint32_t v){ f.write(reinterpret_cast<char*>(&v), 4); };
18     auto put16 = [&](uint16_t v){ f.write(reinterpret_cast<char*>(&v), 2); };
19
20     f.write("RIFF", 4); put32(36 + dataBytes); f.write("WAVE", 4);
21     f.write("fmt ", 4); put32(16); // fmt yigin boyutu
22     put16(1); // PCM
23     put16(channels); put32(sampleRate); put32(byteRate);
24     put16(blockAlign); put16(bitsPerSample);
25     f.write("data", 4); put32(dataBytes);
26
27     for (float s : samples) {

```

```
28     float c = std::fmax(-1.0f, std::fmin(1.0f, s)); // kirp (clip)
29     int16_t q = static_cast<int16_t>(std::lround(c * 32767.0f));
30     f.write(reinterpret_cast<char*>(&q), sizeof(q));
31 }
32 }
33
34 int main() {
35     const double fs = 48000.0;
36     std::vector<float> tone;
37     // 2 saniye, 440 Hz (La notasi), yavasca sonlanan zarfla
38     for (int n = 0; n < static_cast<int>(2 * fs); ++n) {
39         double t = n / fs;
40         double env = std::exp(-1.5 * t); // ussel sonumlenme
41         tone.push_back(static_cast<float>(0.6 * env * std::sin(2*M_PI*440*t)));
42     }
43     writeWav("la440.wav", tone, 48000, 1);
44     return 0;
45 }
```

İpucu

Ürettiğiniz herhangi bir işaretin frekans içeriğini hızlıca görmek için onu WAV'a yazıp Audacity veya sox ile spektrogramına bakabilirsiniz: `sox sinyal.wav -n spectrogram`. Bir demodülatörü hata ayıklarken “kulakla dinlemek” şaşırtıcı derecede güçlü bir araçtır.

KISIM 2

Frekans Alanı: DFT ve FFT

6. Fourier Fikri

İşaret işlemenin en derin fikri şudur: **her işaret, farklı frekanslardaki sinüslerin toplamı olarak yazılabilir.** Zaman alanında karmaşık görünen bir sinyal, frekans alanında birkaç basit tepe hâline gelebilir. Filtreleme, modülasyon, sıkıştırma — çoğu işlem frekans alanında düşünüldüğünde apaçık olur.

Ayrık Fourier Dönüşümü (DFT), N örneklik bir işareti N frekans bileşenine dönüştürür:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$

Buradaki $X[k]$ karmaşıktır: büyüklüğü $|X[k]|$ o frekanstaki bileşenin gücünü, açısı $\angle X[k]$ ise fazını verir. k 'nci bileşenin karşılık geldiği gerçek frekans $f_k = kf_s/N$ 'dir.

6.1 DFT'nin Doğrudan Uygulaması

Tanımı olduğu gibi koda dökmek $O(N^2)$ 'lik bir algoritma verir — yavaştır ama her zaman doğrudur ve FFT'yi doğrulamak için harika bir referanstır:

```

1  #include <complex>
2  #include <vector>
3  #include <cmath>
4
5  using cf64 = std::complex<double>;
6
7  // Doğrudan DFT:  $O(N^2)$ . Girdi gercek veya karmasik olabilir.
8  std::vector<cf64> dft(const std::vector<cf64>& x) {
9      const size_t N = x.size();
10     std::vector<cf64> X(N);
11     for (size_t k = 0; k < N; ++k) {
12         cf64 sum(0.0, 0.0);
13         for (size_t n = 0; n < N; ++n) {
14             double angle = -2.0 * M_PI * k * n / N;
15             sum += x[n] * cf64(std::cos(angle), std::sin(angle));
16         }
17         X[k] = sum;
18     }
19     return X;
20 }
21
22 // Ters DFT (IDFT): frekanstan zamana geri don.
```

```

23 std::vector<cf64> idft(const std::vector<cf64>& X) {
24     const size_t N = X.size();
25     std::vector<cf64> x(N);
26     for (size_t n = 0; n < N; ++n) {
27         cf64 sum(0.0, 0.0);
28         for (size_t k = 0; k < N; ++k) {
29             double angle = +2.0 * M_PI * k * n / N; // isaret ters, +
30             sum += X[k] * cf64(std::cos(angle), std::sin(angle));
31         }
32         x[n] = sum / static_cast<double>(N); // 1/N normalizasyonu
33     }
34     return x;
35 }

```

7. FFT: Hızlı Fourier Dönüşümü

DFT'nin $O(N^2)$ maliyeti büyük N için dayanılmaz olur ($N = 10^6$ için 10^{12} işlem). **FFT**, Cooley–Tukey algoritmasıyla bunu $O(N \log N)$ 'e düşürür — $N = 10^6$ için milyon kat hızlanma. Fikir *böl ve fethet*: N noktalı DFT'yi çift ve tek indeksli örneklerin iki adet $N/2$ noktalı DFT'sine ayır, sonra “kelebek” (butterfly) işlemleriyle birleştir.

7.1 Özyinelemeli Radix-2 FFT

En anlaşılır uygulama özyinelemelidir. N 'in 2'nin kuvveti olması gerekir:

```

1  #include <complex>
2  #include <vector>
3  #include <cmath>
4
5  using cf64 = std::complex<double>;
6
7  // Özyinelemeli radix-2 DIT FFT. x.size() 2'nin kuvveti olmalı.
8  void fftRecursive(std::vector<cf64>& x) {
9      const size_t N = x.size();
10     if (N <= 1) return;
11
12     // Çift ve tek indeksleri ayır
13     std::vector<cf64> even(N/2), odd(N/2);
14     for (size_t i = 0; i < N/2; ++i) {
15         even[i] = x[2*i];
16         odd[i] = x[2*i + 1];
17     }
18     fftRecursive(even);
19     fftRecursive(odd);
20
21     // Kelebek birleştirme
22     for (size_t k = 0; k < N/2; ++k) {
23         double angle = -2.0 * M_PI * k / N;
24         cf64 tw = std::polar(1.0, angle) * odd[k]; // twiddle carpani
25         x[k] = even[k] + tw;
26         x[k + N/2] = even[k] - tw;
27     }
28 }

```

7.2 Yerinde (In-Place) Yinelemeli FFT

Üretim kodu özyineleme yerine, bit-ters (bit-reversal) permütasyon + iç içe döngü kullanır. Bellek tahsisi yapmaz, önbellek dostudur ve çok daha hızlıdır:

```

1  #include <complex>
2  #include <vector>
3  #include <cmath>
4
5  using cf64 = std::complex<double>;
6
7  // Yinelemeli, yerinde radix-2 FFT. invert=true ise ters FFT (1/N'siz).
8  void fft(std::vector<cf64>& a, bool invert = false) {
9      const size_t n = a.size();
10     if (n <= 1) return;
11
12     // 1) Bit-ters permutasyon
13     for (size_t i = 1, j = 0; i < n; ++i) {
14         size_t bit = n >> 1;
15         for (; j & bit; bit >>= 1) j ^= bit;
16         j ^= bit;
17         if (i < j) std::swap(a[i], a[j]);
18     }
19
20     // 2) Kelebekleri katman katman uygula (len = 2, 4, 8, ...)
21     for (size_t len = 2; len <= n; len <= 1) {
22         double ang = (invert ? +2.0 : -2.0) * M_PI / len;
23         cf64 wlen(std::cos(ang), std::sin(ang));
24         for (size_t i = 0; i < n; i += len) {
25             cf64 w(1.0, 0.0);
26             for (size_t k = 0; k < len/2; ++k) {
27                 cf64 u = a[i + k];
28                 cf64 v = a[i + k + len/2] * w;
29                 a[i + k] = u + v;
30                 a[i + k + len/2] = u - v;
31                 w *= wlen;
32             }
33         }
34     }
35
36     if (invert)
37         for (cf64& x : a) x /= static_cast<double>(n); // ters için 1/N
38 }

```

Kullanım: Bir Frekansı Tespit Et

Aşağıdaki program, 8000 Hz'te örneklenmiş ve içinde 1000 Hz'lik bir ton bulunan bir işaretin FFT'sini alıp en güçlü frekans binini bulur.

```

1  #include <cstdio>
2
3  int main() {
4      const double fs = 8000.0;
5      const size_t N = 1024; // 2'nin kuvveti
6      std::vector<cf64> x(N);
7      for (size_t n = 0; n < N; ++n)
8          x[n] = std::sin(2*M_PI*1000.0*n/fs); // gercek girdi, Q=0
9
10     fft(x); // yerinde donusum
11 }

```

```

12 // En büyük büyüklükteki bin'i bul (yalnız ilk yariya bak: gerçek girdi)
13 size_t peak = 0; double best = 0;
14 for (size_t k = 0; k < N/2; ++k) {
15     double mag = std::abs(x[k]);
16     if (mag > best) { best = mag; peak = k; }
17 }
18 printf("Tepe bin = %zu -> %.1f Hz\n", peak, peak * fs / N); // ~1000 Hz
19 return 0;
20 }

```

Frekans çözünürlüğü

FFT'nin bin aralığı $\Delta f = f_s/N$ 'dir. Daha ince frekans çözünürlüğü için ya daha uzun kayıt alırsınız (büyük N) ya da işareti sıfırlarla doldurursunuz (zero-padding). Bin çözünürlüğü zamanla ters orantılıdır: 1 Hz çözünürlük için en az 1 saniyelik kayıt gerekir.

8. Pencereleme (Windowing)

DFT, işaretin *periyodik* olduğunu varsayar. Ama biz sonlu bir parça kesip aldığımızda, kesme noktalarındaki süreksizlik frekans alanında **spektral sızıntıya** (spectral leakage) yol açar: keskin bir tepe olması gereken yerde, yanlara yayılan yan loblar (sidelobes) oluşur. Çözüm, işareti kenarlarda yumuşakça sıfıra indiren bir **pencere fonksiyonuyla** çarpmaktır.

Pencere	Özelliği
Dikdörtgen (yok)	En dar ana lob, en kötü sızıntı. Geçici olayları ölçmek için.
Hann	İyi genel amaçlı denge; yumuşak, -31 dB yan lob.
Hamming	Hann'a benzer, ilk yan lobu bastırır (-43 dB).
Blackman	Çok düşük yan lob (-58 dB), daha geniş ana lob. Zayıf tonları görmek için.

```

1 #include <vector>
2 #include <cmath>
3
4 // Hann penceresi: w[n] = 0.5 * (1 - cos(2*pi*n/(N-1)))
5 std::vector<double> hannWindow(size_t N) {
6     std::vector<double> w(N);
7     for (size_t n = 0; n < N; ++n)
8         w[n] = 0.5 * (1.0 - std::cos(2.0 * M_PI * n / (N - 1)));
9     return w;
10 }
11
12 // Blackman penceresi: daha iyi yan-lob bastırma
13 std::vector<double> blackmanWindow(size_t N) {
14     std::vector<double> w(N);
15     for (size_t n = 0; n < N; ++n) {
16         double t = 2.0 * M_PI * n / (N - 1);
17         w[n] = 0.42 - 0.5 * std::cos(t) + 0.08 * std::cos(2 * t);
18     }
19     return w;
20 }
21

```

```

22 // Spektrum almadan önce pencereyi uygula.
23 void applyWindow(std::vector<cf64>& x, const std::vector<double>& w) {
24     for (size_t n = 0; n < x.size(); ++n) x[n] *= w[n];
25 }

```

8.1 Güç Spektral Yoğunluğu (PSD)

Bir işaretin gücünün frekansa nasıl dağıldığını görmek için PSD hesaplarız. Pratikte pencere-
lenmiş FFT'nin büyüklük karesini alıp desibele çeviririz. SDR programlarındaki “şelale” (waterfall)
görüntüsünün altında yatan hesap budur:

```

1  #include <vector>
2  #include <cmath>
3
4  // Tek bir kare için dB cinsinden güç spektrumu.
5  std::vector<double> powerSpectrumDb(std::vector<cf64> x,
6                                     const std::vector<double>& win) {
7
8      const size_t N = x.size();
9      applyWindow(x, win);
10     fft(x); // (önceki bölümdeki FFT)
11
12     // Pencere kazancını telafi et
13     double winSum = 0; for (double w : win) winSum += w;
14
15     std::vector<double> psd(N);
16     for (size_t k = 0; k < N; ++k) {
17         double mag = std::abs(x[k]) / winSum;
18         psd[k] = 20.0 * std::log10(mag + 1e-12); // +eps: log(0)'dan kaçın
19     }
20     return psd;
21 }

```

KISIM 3

Filtreler: FIR ve IIR

9. Konvolüsyon ve FIR Filtreler

Bir **filtre**, işaretin belli frekanslarını geçirip diğerlerini zayıflatan bir sistemdir. En temel yapı taşı **konvolüsyondur**: çıktı, girdinin filtre *dürtü yanıtı* $h[n]$ ile konvolüsyonudur:

$$y[n] = \sum_{k=0}^{M-1} h[k] x[n-k]$$

Dürtü yanıtı sonlu uzunluktaysa (yani h sonlu bir katsayı dizisiyse) buna **FIR** (Finite Impulse Response) filtre denir. FIR filtreler her zaman kararlıdır ve **tam doğrusal faz** sağlayabilir (tüm frekansları aynı gecikmeyle geciktirir; ses ve haberleşmede kritik).

9.1 Temel FIR Uygulaması

```

1  #include <vector>
2
3  // Zamanla akan bir FIR filtresi. Katsayılar (taps) sabit; her örnek için
4  // geçmiş M örneklilik pencereyi bir dairesel tampon (ring buffer) ile tutar.
5  class FirFilter {
6      std::vector<float> taps_;    // h[k] katsayıları
7      std::vector<float> buf_;    // geçmiş örnekler (dairesele)
8      size_t pos_ = 0;
9  public:
10     explicit FirFilter(std::vector<float> taps)
11         : taps_(std::move(taps)), buf_(taps_.size(), 0.0f) {}
12
13     // Bir örnek işle, bir örnek döndür.
14     float process(float x) {
15         buf_[pos_] = x;
16         float acc = 0.0f;
17         size_t idx = pos_;
18         for (size_t k = 0; k < taps_.size(); ++k) {
19             acc += taps_[k] * buf_[idx];
20             idx = (idx == 0) ? taps_.size() - 1 : idx - 1; // geriye sar
21         }
22         pos_ = (pos_ + 1) % taps_.size();
23         return acc;
24     }
25
26     void reset() { std::fill(buf_.begin(), buf_.end(), 0.0f); pos_ = 0; }

```


27 };

9.2 Pencere Yöntemiyle Alçak Geçiren FIR Tasarımı

Peki katsayıları nereden buluruz? İdeal bir alçak geçiren filtrenin dürtü yanıtı bir **sinc** fonksiyonudur. Onu sonlu uzunlukta kesip bir pencereyle yumuşatarak pratik bir filtre elde ederiz:

```

1  #include <vector>
2  #include <cmath>
3
4  // Pencere-sinc yontemiyle alcak geciren FIR tasarla.
5  //  cutoffHz : -6 dB kesim frekansi
6  //  fs       : ornekleme frekansi
7  //  numTaps  : katsayi sayisi (tek sayi onerilir -> simetrik, tam dogrusal faz)
8  std::vector<float> designLowpass(double cutoffHz, double fs, size_t numTaps) {
9      std::vector<float> h(numTaps);
10     const double fc = cutoffHz / fs;           // normalize kesim (0..0.5)
11     const double mid = (numTaps - 1) / 2.0;
12     double sum = 0.0;
13
14     for (size_t n = 0; n < numTaps; ++n) {
15         double m = n - mid;
16         // Ideal alcak geciren dPurtu yaniti: sinc
17         double sinc = (m == 0.0) ? 2.0 * fc
18                        : std::sin(2.0 * M_PI * fc * m) / (M_PI * m);
19         // Hamming penceresi
20         double win = 0.54 - 0.46 * std::cos(2.0 * M_PI * n / (numTaps - 1));
21         h[n] = static_cast<float>(sinc * win);
22         sum += h[n];
23     }
24     // DC'de birim kazanc icin normalize et
25     for (float& c : h) c /= static_cast<float>(sum);
26     return h;
27 }
28
29 // Yuksek geciren = all-pass eksi alcak geciren (spektral tersleme)
30 std::vector<float> designHighpass(double cutoffHz, double fs, size_t numTaps) {
31     auto h = designLowpass(cutoffHz, fs, numTaps);
32     for (float& c : h) c = -c;           // isaretini ters cevir
33     h[(numTaps - 1) / 2] += 1.0f;       // merkeze birim dPurtu ekle
34     return h;
35 }

```

Uygulama: Gürültülü Sinyali Temizle

500 Hz'lik yararlı tonu 5000 Hz'lik gürültüden ayıralım.

```

1  int main() {
2      const double fs = 44100.0;
3      // 500 Hz sinyal + 5000 Hz gurultu
4      std::vector<float> in;
5      for (int n = 0; n < 4410; ++n) {
6          double t = n / fs;
7          in.push_back(0.5*std::sin(2*M_PI*500*t) + 0.5*std::sin(2*M_PI*5000*t));
8      }
9      // 1500 Hz'te kesen 101-tap alcak geciren filtre
10     FirFilter lp(designLowpass(1500.0, fs, 101));
11     std::vector<float> out;

```

```

12     for (float x : in) out.push_back(lp.process(x));
13     // out: yalnızca 500 Hz kalır; 5000 Hz ~40 dB bastırılmış olur.
14     return 0;
15 }

```

10. IIR Filtreler ve Biquad

IIR (Infinite Impulse Response) filtreler geri besleme kullanır: çıktı hem geçmiş girdilere hem geçmiş *çıkıtlara* bağlıdır. Bu sayede çok az katsayıyla çok keskin filtreler kurulabilir — ama kararlılık artık garanti değildir ve faz doğrusal olmaz. En yaygın yapı taşı **biquad**'dır (ikinci dereceden bölüm):

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]$$

```

1  #include <cmath>
2
3  // Transpoze Direct Form II biquad (sayısal olarak daha kararlı).
4  struct Biquad {
5      float b0=1, b1=0, b2=0, a1=0, a2=0;    // katsayılar (a0=1'e normalize)
6      float z1=0, z2=0;                    // durum degiskenleri
7
8      float process(float x) {
9          float y = b0 * x + z1;
10         z1 = b1 * x - a1 * y + z2;
11         z2 = b2 * x - a2 * y;
12         return y;
13     }
14     void reset() { z1 = z2 = 0; }
15
16     // RBJ "Audio EQ Cookbook" katsayılarıyla alacak geciren biquad kur.
17     static Biquad lowpass(double freqHz, double fs, double Q = 0.7071) {
18         Biquad bq;
19         double w0 = 2.0 * M_PI * freqHz / fs;
20         double cw = std::cos(w0), sw = std::sin(w0);
21         double alpha = sw / (2.0 * Q);
22         double a0 = 1 + alpha;
23         bq.b0 = (1 - cw) / 2 / a0;
24         bq.b1 = (1 - cw) / a0;
25         bq.b2 = (1 - cw) / 2 / a0;
26         bq.a1 = (-2 * cw) / a0;
27         bq.a2 = (1 - alpha) / a0;
28         return bq;
29     }
30     // Band geciren (tepe kazancli) biquad
31     static Biquad bandpass(double freqHz, double fs, double Q = 5.0) {
32         Biquad bq;
33         double w0 = 2.0 * M_PI * freqHz / fs;
34         double cw = std::cos(w0), sw = std::sin(w0);
35         double alpha = sw / (2.0 * Q);
36         double a0 = 1 + alpha;
37         bq.b0 = alpha / a0; bq.b1 = 0; bq.b2 = -alpha / a0;
38         bq.a1 = (-2 * cw) / a0; bq.a2 = (1 - alpha) / a0;
39         return bq;
40     }
41 };

```

Dikkat

IIR filtreler tasarımı kolay ama tuzaklıdır. Katsayılar kutupları (poles) birim çemberin *dışına* taşarsa filtre patlar (kararsız olur). Yüksek dereceli IIR'ları tek bir denklemle değil, art arda bağlı biquad'larla (cascade) uygulayın — bu, sayısal hata birikimini ve kararsızlığı ciddi biçimde azaltır.

10.1 Hareketli Ortalama ve CIC: Ucuz Filtreler

En ucuz alçak geçiren filtre **hareketli ortalama**dır. Onu bir toplam-tut (running sum) ile $O(1)$ 'de uygulayabiliriz — SDR'da desimasyon öncesi kaba filtreleme için idealdir:

```

1  #include <vector>
2
3  // O(1) hareketli ortalama filtresi (kutu filtre / boxcar).
4  class MovingAverage {
5      std::vector<float> buf_;
6      size_t pos_ = 0, count_ = 0;
7      float sum_ = 0.0f;
8  public:
9      explicit MovingAverage(size_t window) : buf_(window, 0.0f) {}
10     float process(float x) {
11         sum_ -= buf_[pos_];           // pencereden cikani cikar
12         buf_[pos_] = x;
13         sum_ += x;                   // yeni gireni ekle
14         pos_ = (pos_ + 1) % buf_.size();
15         if (count_ < buf_.size()) ++count_;
16         return sum_ / count_;
17     }
18 };

```

KISIM 4

IQ ve Karmaşık Bant Temeli

11. IQ Nedir? Analitik İşaret

Şimdi tüm modern haberleşmenin kalbine geliyoruz. **IQ**, “In-phase” (I, eş-fazlı) ve “Quadrature” (Q, dik-fazlı, 90° kaymış) bileşenlerin kısa adıdır. Bir IQ örneği aslında bir **karmaşık sayıdır**: $z = I + jQ$.

Neden gerçek bir sinüs yerine karmaşık bir sayı? Çünkü gerçek bir sinüs $\cos(2\pi ft)$, frekans alanında hem $+f$ hem $-f$ 'te iki tepe içerir — yani pozitif ve negatif frekansı ayırt edemez. Oysa karmaşık üstel $e^{j2\pi ft}$ yalnızca $+f$ 'te tek bir tepe içerir. Bu “tek taraflılık” sayesinde:

- Bir sinyalin frekansının *işaretini* (yönünü) bilebiliriz.
- Frekansı yukarı/aşağı kaydırırken görüntü (image) frekansı oluşmaz.
- Genlik, faz ve frekansı aynı anda, ayrı ayrı çekebiliriz.

Bilgi

Bir SDR alıcısı antenden gelen yüksek frekanslı gerçek işareti, donanımda iki mikserle (biri $\cos$, biri $\sin$ ile) çarpıp I ve Q akışlarını üretir. Yazılıma ulaşan şey artık taşıyıcıdan arındırılmış, **karmaşık bant temeli** (complex baseband) işaretidir: 0 Hz civarına indirilmiş, hem pozitif hem negatif frekansları temsil edebilen bir `complex<float>` akışı.

11.1 Analitik İşaret ve Hilbert Dönüşümü

Elimizde yalnızca gerçek bir işaret varsa (örneğin bir ses kaydı), onun karmaşık “analitik” karşılığını üretebiliriz. Analitik işaret, negatif frekansları sıfırlanmış hâlidir: $x_a[n] = x[n] + j\hat{x}[n]$, burada $\hat{x}$ işaretin **Hilbert dönüşümüdür** (90° faz kaymış hâli). Bunun en temiz yolu FFT üzerinden gider:

```
1 #include <complex>
2 #include <vector>
3
4 using cf64 = std::complex<double>;
5
6 // Gerçek bir işaretin analitik (karmaşık) karşılığını üret.
7 // Negatif frekansları sıfırlayıp pozitifleri 2'yle carparak.
8 std::vector<cf64> analyticSignal(const std::vector<double>& x) {
9     const size_t N = x.size();
10    std::vector<cf64> X(N);
11    for (size_t n = 0; n < N; ++n) X[n] = cf64(x[n], 0.0);
```

```

12     fft(X);                                // frekans alanına geç
13
14     // Hilbert filtresi: DC ve Nyquist aynı; pozitif frekanslar x2; negatifler 0
15     for (size_t k = 1; k < N/2; ++k)        X[k]      *= 2.0;
16     for (size_t k = N/2 + 1; k < N; ++k)    X[k]      = 0.0;
17
18     fft(X, /*invert=*/true);                // zamana geri dön
19     return X;                               // artık analitik (karmaşık)
20 }
21

```

İpucu

Analitik işaretin büyüklüğü $|x_a[n]|$ işaretin **anlık zarfını** (envelope), açısı ise **anlık fazını** verir. Zarf, AM demodülasyonunun; fazın türevi ise FM demodülasyonunun temelidir. Yani bir kez karmaşık bant temeline geçtiğinizde, tüm demodülatörler birkaç satır kalır.

12. NCO, Mikser ve Frekans Kaydırma

Bir işareti frekans ekseninde kaydırmak (mikslemek), onu bir karmaşık üstel ile çarpmaktır. Bu üsteli üreten yapıya **NCO** (Numerically Controlled Oscillator) denir. Bir işareti $-f_0$ kadar kaydırmak (yani f_0 'daki bir kanalı DC'ye indirmek) için $e^{-j2\pi f_0 n/f_s}$ ile çarparsınız:

$$y[n] = x[n] \cdot e^{-j2\pi f_0 n/f_s}$$

```

1  #include <complex>
2  #include <vector>
3  #include <cmath>
4
5  using cf32 = std::complex<float>;
6
7  // Sayısal kontrollü osilator (NCO). Her çağrıda bir sonraki fazor örneğini verir.
8  // Faz'ı tamsayı bir birikimle değil, kayan noktali ama sarmalanmış tutar.
9  class Nco {
10     double phase_ = 0.0;           // anlık faz (rad)
11     double phaseInc_;              // örnek başına faz artışı
12 public:
13     Nco(double freqHz, double fs)
14         : phaseInc_(2.0 * M_PI * freqHz / fs) {}
15
16     void setFrequency(double freqHz, double fs) {
17         phaseInc_ = 2.0 * M_PI * freqHz / fs;
18     }
19
20     cf32 next() {
21         cf32 out(std::cos(phase_), std::sin(phase_));
22         phase_ += phaseInc_;
23         // Fazı [-pi, pi] aralığında tut (sayısal hata birikimini onle)
24         if (phase_ > M_PI) phase_ -= 2.0 * M_PI;
25         if (phase_ < -M_PI) phase_ += 2.0 * M_PI;
26         return out;
27     }
28 };
29
30 // Bir karmaşık bant temeli akisini freqHz kadar kaydır (mikser).

```

```

31 void frequencyShift(std::vector<cf32>& iq, double freqHz, double fs) {
32     Nco lo(freqHz, fs);           // negatif verirken asagi kaydirir
33     for (cf32& s : iq) s *= lo.next();
34 }

```

Kanal Seçimi

Diyelim ki 2.4 MHz genişlikte bir IQ akışında, merkezden +300 kHz ötede bir istasyon var. Onu tam ortaya (DC'ye) getirmek için akışı -300 kHz kaydırırız; sonra alçak geçiren filtreleyip yalnız o kanalı bırakırız.

```

1 void tuneToChannel(std::vector<cf32>& iq, double offsetHz, double fs) {
2     frequencyShift(iq, -offsetHz, fs); // istenen kanali DC'ye getir
3     // Sonrasında: alçak geciren filtre + desimasyon (asagidaki bolum)
4 }

```

Verimlilik

Her örnekte sin/cos çağırmak pahalıdır. Üretim kodunda NCO ya bir arama tablosu (LUT) kullanır ya da bir başlangıç fazörünü her adımda sabit bir cf32 çarpanıyla döndürür: `osc *= step;` İkincisi çok hızlıdır ama fazörün büyüklüğü zamanla sürüklenir, bu yüzden ara sıra `osc /= std::abs(osc)` ile yeniden normalize edilir.

13. Desimasyon ve Örneklem Hızı Dönüşümü

Bir SDR akışı genelde 2–10 MHz hızındadır ama biz sonunda 48 kHz'lik ses istiyoruz. **Desimasyon** (decimation), örneklem hızını M kat düşürmektir: önce alçak geçiren filtre (yeni Nyquist üstünü at, yoksa aliasing!), sonra her M örnekten birini tut.

```

1 #include <vector>
2
3 // M kat desimasyon: once anti-alias filtre, sonra her M'inci ornegi al.
4 class Decimator {
5     FirFilter aa_;           // anti-aliasing alçak geciren (onceki bolum)
6     size_t M_, phase_ = 0;
7 public:
8     Decimator(size_t M, std::vector<float> taps)
9         : aa_(std::move(taps)), M_(M) {}
10
11     // Girdi bloklarini isle; cikti girdinin ~1/M kadar orneginidir.
12     std::vector<float> process(const std::vector<float>& in) {
13         std::vector<float> out;
14         out.reserve(in.size() / M_ + 1);
15         for (float x : in) {
16             float y = aa_.process(x); // her ornek filtreden gecer
17             if (phase_ == 0) out.push_back(y);
18             phase_ = (phase_ + 1) % M_; // ama yalnız 1/M'i saklanir
19         }
20         return out;
21     }
22 };

```

Bilgi

Verimlilik ipucu: Filtre çıktısının çoğunu atacaksak, onları hesaplamak israftır. *Polifaz desimasyon* yalnızca tutulacak örnekleri hesaplayarak M kat hızlanma sağlar. Ayrıca çok büyük desimasyon oranları için (örn. $M = 100$) çarpma içermeyen **CIC filtreleri** kullanılır — bunlar SDR donanımının FPGA'sında sıkça bulunur.

13.1 Karmaşık (IQ) Desimasyon

Yukarıdaki FirFilter gerçek örnekler içindi. IQ için I ve Q kanallarını ayrı ayrı filtreleriz. Pratikte tek bir karmaşık FIR yazmak daha temizdir:

```

1  #include <complex>
2  #include <vector>
3
4  using cf32 = std::complex<float>;
5
6  // Karmaşık girdi, gercek katsayılı FIR (kanal filtresi için tipik).
7  class ComplexFir {
8      std::vector<float> taps_;
9      std::vector<cf32> buf_;
10     size_t pos_ = 0;
11 public:
12     explicit ComplexFir(std::vector<float> taps)
13         : taps_(std::move(taps)), buf_(taps_.size(), cf32(0,0)) {}
14
15     cf32 process(cf32 x) {
16         buf_[pos_] = x;
17         cf32 acc(0, 0);
18         size_t idx = pos_;
19         for (size_t k = 0; k < taps_.size(); ++k) {
20             acc += taps_[k] * buf_[idx];
21             idx = (idx == 0) ? taps_.size() - 1 : idx - 1;
22         }
23         pos_ = (pos_ + 1) % taps_.size();
24         return acc;
25     }
26 };

```

KISIM 5

Modülasyon ve Demodülasyon

14. Genel Bakış: Modülasyon Neden Var?

Modülasyon, taşımak istediğimiz bilgiyi (mesaj işareti) yüksek frekanslı bir *taşıyıcıya* (carrier) bindirmektir. Buna neden ihtiyaç var?

- **Anten boyutu:** Verimli bir anten dalga boyunun çeyreği kadardır. 1 kHz’lik ses için bu ~ 75 km’lik bir anten demektir — imkânsız. 100 MHz taşıyıcıya bindirirsek anten ~ 75 cm olur.
- **Çoklama:** Farklı istasyonlar farklı taşıyıcı frekanslarını kullanarak aynı havayı paylaşabilir (frekans bölmeli çoklama).
- **Gürültü dayanıklılığı:** FM gibi açı modülasyonları, genlik gürültüsüne karşı doğal bir bağışıklık sağlar.

Bilgiyi taşıyıcının hangi özelliğine gömdüğümüze göre üç temel aile vardır:

Aile	Değişen özellik	Örnekler
Genlik	taşıyıcı genliği $A(t)$	AM, DSB, SSB, ASK, QAM
Frekans	anlık frekans $f(t)$	FM, FSK
Faz	anlık faz $\phi(t)$	PM, BPSK, QPSK, PSK

Karmaşık bant temelinde bir modüle işaret genel olarak şöyle yazılır:

$$s(t) = A(t) e^{j\phi(t)}$$

Genlik modülasyonu $A(t)$ ’yi, açı modülasyonları (FM/PM) $\phi(t)$ ’yi bilgiyle oynar. Bütün demodülatörlerin işi, bu karmaşık işaretten $A(t)$ veya $\phi(t)$ ’yi geri çekmektir — ki bunu Kısım 4’teki `std::abs` ve `std::arg` ile yapacağımızı zaten sezdimiz.

15. AM: Genlik Modülasyonu

En eski ve en basit modülasyon. Mesaj $m(t)$, taşıyıcının genliğini oynatır:

$$s(t) = (1 + \mu m(t)) \cos(2\pi f_c t)$$

Buradaki μ **modülasyon indeksi**dir (genelde ≤ 1 ; aşarsa “aşırı modülasyon” ve bozulma olur). “1+” terimi taşıyıcının kendisini de gönderir — bu, alıcıda basit bir zarf detektörü kullanmayı mümkün kılar (bu yüzden eski radyolar ucuzdu).

AM’in Spektrumu ve Verimliliği

Tek tonlu $m(t) = \cos(2\pi f_m t)$ için çarpım terimini açalım. Trigonometrik çarpım-toplam özdeşliği $\cos A \cos B = \frac{1}{2}[\cos(A-B) + \cos(A+B)]$ ’yi kullanırsak:

$$s(t) = \underbrace{\cos(2\pi f_c t)}_{\text{taşıyıcı}} + \underbrace{\frac{\mu}{2} \cos(2\pi(f_c - f_m)t)}_{\text{alt yan bant}} + \underbrace{\frac{\mu}{2} \cos(2\pi(f_c + f_m)t)}_{\text{üst yan bant}}$$

Yani AM spektrumu taşıyıcı f_c ile onun $\pm f_m$ yanındaki iki yan banttan oluşur. Bant genişliği $B = 2f_m$ ’dir — mesajın iki katı. Bilgi *yalnız* yan bantlardadır; taşıyıcı hiçbir bilgi taşımaz ama tüm gücün çoğunu tüketir. $\mu = 1$ ’de bile güç verimliliği sadece $\frac{1}{3}$ ’tür. Bu israfı gidermek için taşıyıcı bastırılır (**DSB-SC**) veya bir yan bant da atılır (**SSB**, yarı bant genişliği) — ama bunlar alıcıda senkron demodülasyon zorunlu kılar.

15.1 AM Modülatörü

```

1 #include <vector>
2 #include <cmath>
3
4 // Klasik çift yan bantlı, taşıyıcılı AM (DSB-LC).
5 std::vector<float> amModulate(const std::vector<float>& msg,
6                               double carrierHz, double fs, double mu = 0.8) {
7     std::vector<float> out(msg.size());
8     for (size_t n = 0; n < msg.size(); ++n) {
9         double carrier = std::cos(2.0 * M_PI * carrierHz * n / fs);
10        out[n] = static_cast<float>((1.0 + mu * msg[n]) * carrier);
11    }
12    return out;
13 }
```

15.2 Zarf (Envelope) Detektörü ile Demodülasyon

En basit AM demodülasyonu işaretin zarfını takip etmektir. Klasik analog devre diyot + RC filtreydi; sayısalda ise ya analitik işaretin büyüklüğünü ($|x_a|$) alırız ya da doğrudan tam-dalga doğrultup alçak geçiren filtreleriz:

```

1 #include <vector>
2 #include <cmath>
3
4 // Yöntem 1: Analitik işaretin büyüklüğü = anlık zarf (en temiz).
5 std::vector<float> amEnvelopeDemod(const std::vector<double>& rf) {
6     auto analytic = analyticSignal(rf); // Kisim 4'teki fonksiyon
7     std::vector<float> msg(analytic.size());
8     for (size_t n = 0; n < analytic.size(); ++n)
9         msg[n] = static_cast<float>(std::abs(analytic[n]) - 1.0); // DC'yi çıkar
10    return msg;
11 }
12
13 // Yöntem 2: Doğrultma + alçak geçiren (klasik "diyot detektör" dijital hali).
14 std::vector<float> amRectifierDemod(const std::vector<float>& rf,
15                                     double cutoffHz, double fs) {
16     Biquad lp = Biquad::lowpass(cutoffHz, fs);
17     std::vector<float> msg(rf.size());
18     for (size_t n = 0; n < rf.size(); ++n)
```

```

19     msg[n] = lp.process(std::fabs(rf[n])); // tam-dalga dogrultma
20     // Kalan DC bileşeni (tasiyici) ayrıca yuksek geciren ile atilabilir.
21     return msg;
22 }

```

15.3 Senkron (Coherent) AM Demodülasyonu

Zarf detektörü basittir ama gürültüde ve aşırı modülasyonda bozulur. Daha iyisi, alıcıda taşıyıcının aynısını üretip işaretle çarpma (product detector). Bant temeline inmiş IQ akışında bu, sadece **I** bileşenini almaktır:

```

1  #include <complex>
2  #include <vector>
3
4  using cf32 = std::complex<float>;
5
6  // IQ bant temelinde AM: mesaj, tasiyici fazina hizalanmis I bileşenidir.
7  // (Taban IQ'nun tasiyicisi zaten DC'ye indirilmis kabul edilir.)
8  std::vector<float> amCoherentDemod(const std::vector<cf32>& iq) {
9      std::vector<float> msg(iq.size());
10     for (size_t n = 0; n < iq.size(); ++n)
11         msg[n] = iq[n].real(); // I = genlik bilgisi
12     return msg;
13 }

```

Dikkat

Senkron demodülasyon, alıcının taşıyıcı fazını *tam olarak* bilmesini gerektirir. Gerçekte taşıyıcı fazı bilinmez ve sürüklenir; bu yüzden onu bir **PLL** (Faz Kilitlemeli Döngü, Kısım 5.8) ile takip ederiz. PLL kilitlenene kadar I ve Q karışır ve ses cılız çıkar. Bu, tüm senkron alıcıların ortak sorunudur.

16. FM: Frekans Modülasyonu

FM, bilgiyi taşıyıcının **anlık frekansına** gömer. Anlık frekans, mesajla orantılı olarak nominal değerinin etrafında sapar:

$$f_i(t) = f_c + k_f m(t)$$

Faz, frekansın integrali olduğundan, modüle işaret:

$$s(t) = A \cos\left(2\pi f_c t + 2\pi k_f \int_0^t m(\tau) d\tau\right)$$

k_f **frekans sapmasını** (deviation) belirler. FM radyo yayınında tepe sapma ± 75 kHz'tir; dar bant FM (telsizler) ± 2.5 kHz kullanır.

16.1 FM'in Matematik: Modülasyon İndeksi, Bessel ve Carson

Tek tonlu bir mesaj $m(t) = \cos(2\pi f_m t)$ ele alalım. Tepe frekans sapması $\Delta f = k_f$ olsun. Modüle işaret:

$$s(t) = A \cos\left(2\pi f_c t + \underbrace{\frac{\Delta f}{f_m}}_{\beta} \sin(2\pi f_m t)\right)$$

Buradaki boyutsuz $\beta = \Delta f / f_m$ niceliğine **modülasyon indeksi** denir ve FM'in karakterini belirler. İşareti açmak için karmaşık gösterime geçelim ve **Jacobi–Anger açılımını** kullanalım:

$$e^{j\beta \sin \theta} = \sum_{k=-\infty}^{\infty} J_k(\beta) e^{jk\theta}$$

Burada $J_k(\beta)$, birinci tür **Bessel fonksiyonudur**. Bunu $s(t)$ 'ye uygularsak:

$$s(t) = A \sum_{k=-\infty}^{\infty} J_k(\beta) \cos(2\pi(f_c + kf_m)t)$$

Bu çarpıcı bir sonuçtur: *tek* bir ton bile FM'de, taşıyıcının etrafında f_m aralıklarla dizilmiş *sonsuz* sayıda yan bant üretir. Her yan bantın genliği $J_k(\beta)$ ile belirlenir. Pratikte $|k| > \beta + 1$ için J_k hızla sıfıra gider, yani bant genişliği sonlu kabul edilebilir. Bu gözlem **Carson kuralını** verir — FM işaretinin gücünün %98'ini içeren yaklaşık bant genişliği:

$$B \approx 2(\Delta f + f_m) = 2f_m(\beta + 1)$$

FM radyo için $\Delta f = 75$ kHz ve $f_m = 15$ kHz alırsak $B \approx 2(75 + 15) = 180$ kHz olur — FM kanallarının neden 200 kHz aralıkla yerleştirildiğinin sebebi budur.

Genlik gürültüsüne bağışıklık

FM'in bilgisi *frekansta* (yani fazın türevinde) olduğu için, genlik üzerine binen gürültü demod-
 ülatörden geçmez — alıcıda önce bir sınırlayıcı (limiter) genliği sabitler. Bu, FM'in AM'e kıyasla
 neden çok daha temiz ses verdiğinin matematiksel kökenidir: SNR, modülasyon indeksinin *karesiyle*
 ($\propto \beta^2$) iyileşir; buna “FM kazancı” denir.

16.2 FM Modülatörü

Anahtar nokta: mesajı *integre edip* faza koymak. Ayrık zamanda integral, birikimli toplamdır:

```

1 #include <vector>
2 #include <cmath>
3
4 // FM modulator. deviationHz: tepe frekans sapması. Mesaj [-1,1] varsayılır.
5 std::vector<float> fmModulate(const std::vector<float>& msg,
6                               double carrierHz, double deviationHz, double fs) {
7     std::vector<float> out(msg.size());
8     double phase = 0.0;
9     const double dt = 1.0 / fs;
10    for (size_t n = 0; n < msg.size(); ++n) {
11        double fInst = carrierHz + deviationHz * msg[n]; // anlık frekans
12        phase += 2.0 * M_PI * fInst * dt;                // fazi biriktir
13        if (phase > M_PI) phase -= 2.0 * M_PI;
14        if (phase < -M_PI) phase += 2.0 * M_PI;
15        out[n] = static_cast<float>(std::cos(phase));
16    }
17    return out;
18 }
19
20 // Karmaşık bant temelli FM (SDR üreticileri böyle yapar): tasiyicisiz.
21 std::vector<cf32> fmModulateBaseband(const std::vector<float>& msg,
22                                       double deviationHz, double fs) {
23     std::vector<cf32> out(msg.size());
24     double phase = 0.0;
25     for (size_t n = 0; n < msg.size(); ++n) {
26         phase += 2.0 * M_PI * deviationHz * msg[n] / fs;

```

```

27     out[n] = cf32(std::cos(phase), std::sin(phase));    // e^(j*phase)
28 }
29 return out;
30 }

```

16.3 FM Demodülasyonu: IQ Diskriminatör

İşte belgenin başında sorduğunuz kısım — ama artık tüm altyapı elimizde olduğu için nefis derecede basit. FM’de bilgi, fazın *türevindedir* (çünkü frekans = fazın türevi). Karmaşık bant temelinde anlık faz $\angle z[n]$ ’dir; onun türevi ardışık iki örneğin faz farkıdır. Bunu hesaplamamanın en zarif yolu **polar diskriminatördür**:

$$m[n] \propto \angle(z[n] \cdot z^*[n-1])$$

Yani ardışık iki IQ örneğinin, birinin eşleniğiyle çarpımının açısı. Bu çarpım açıları çıkardığı için $(\angle z[n] - \angle z[n-1])$ tam da istediğimiz faz farkını verir — üstelik atan2 sarmalanma (wrap) sorunlarını doğal olarak halleder:

```

1  #include <complex>
2  #include <vector>
3
4  using cf32 = std::complex<float>;
5
6  // Polar diskriminator FM demodulatoru. Girdi: karmaşık bant temeli IQ.
7  // Çikti: demodüle mesaj (frekans sapmasıyla orantili).
8  class FmDemod {
9      cf32 prev_{1.0f, 0.0f};    // z[n-1]
10     float gain_;
11 public:
12     // gain, cikti seviyesini normalize eder: fs / (2*pi*deviation)
13     explicit FmDemod(double fs, double deviationHz)
14         : gain_(static_cast<float>(fs / (2.0 * M_PI * deviationHz))) {}
15
16     float process(cf32 z) {
17         cf32 prod = z * std::conj(prev_);    // aci farki = arg(prod)
18         prev_ = z;
19         return std::arg(prod) * gain_;    // atan2 ile faz farki
20     }
21
22     std::vector<float> process(const std::vector<cf32>& iq) {
23         std::vector<float> out(iq.size());
24         for (size_t n = 0; n < iq.size(); ++n) out[n] = process(iq[n]);
25         return out;
26     }
27 };

```

Neden atan2 değil de eşlenik çarpımı?

Doğrudan $\angle z[n] - \angle z[n-1]$ hesaplamak da mümkündür ama faz $\pm\pi$ sınırından geçtiğinde 2π ’lik sıçramalar oluşur; bunları elle “unwrap” etmek gerekir. Eşlenik çarpım yöntemi bu farkı *tek bir atan2 çağrısıyla*, $[-\pi, \pi]$ aralığında, sıçramasız verir. Bu yüzden GNU Radio’nun quadrature_demod bloğu da tam olarak bunu yapar.

Hesaplamadan kaçınmak

Her örnekte `std::arg` (yani `atan2`) çağırmak pahalıdır. Yüksek hızlı alıcılarda bunun yerine yaklaşık formül kullanılır: küçük faz farkları için $\angle(I + jQ) \approx Q/|z|$, hatta çarpım sonucunun $\frac{I\Delta Q - Q\Delta I}{I^2 + Q^2}$ biçimi. Ama başlangıç için `std::arg` hem doğru hem okunurdur — önce çalıştırın, sonra hızlandırın.

16.4 De-emphasis: FM'in Son Rötuşu

FM yayıncılığında, verici tarafta yüksek frekanslar bilerek yükseltilir (pre-emphasis) çünkü gürültü orada daha baskındır. Alıcıda bunu tersine çeviren bir **de-emphasis** filtresi (basit bir RC alçak geçiren) uygularız. ABD'de zaman sabiti 75 μ s, Avrupa'da 50 μ s'tir:

```
1 #include <cmath>
2
3 // Tek kutuplu de-emphasis filtresi (RC alçak geciren).
4 class Deemphasis {
5     float alpha_, prev_ = 0.0f;
6 public:
7     // tauSec: 75e-6 (ABD) veya 50e-6 (Avrupa/Türkiye)
8     Deemphasis(double tauSec, double fs) {
9         double dt = 1.0 / fs;
10        alpha_ = static_cast<float>(dt / (tauSec + dt));
11    }
12    float process(float x) {
13        prev_ += alpha_ * (x - prev_);           // ussel yumusatma
14        return prev_;
15    }
16 };
```

17. PM: Faz Modülasyonu

Faz modülasyonu, mesajı doğrudan faza koyar (integre etmeden):

$$s(t) = A \cos(2\pi f_c t + k_p m(t))$$

FM ile yakın akrabadır: PM'e $m(t)$ 'nin türevini verirse FM elde ederiz, ya da FM'e integralini verirse PM. Demodülasyon FM'e çok benzer ama faz farkını *integre etmeyiz* — doğrudan anlık fazi okuruz:

```
1 #include <complex>
2 #include <vector>
3
4 using cf32 = std::complex<float>;
5
6 // PM modulator (bant temeli): mesaj doğrudan faz.
7 std::vector<cf32> pmModulateBaseband(const std::vector<float>& msg, float kp) {
8     std::vector<cf32> out(msg.size());
9     for (size_t n = 0; n < msg.size(); ++n) {
10        float phase = kp * msg[n];
11        out[n] = cf32(std::cos(phase), std::sin(phase));
12    }
13    return out;
14 }
15
16 // PM demodulator: anlık fazi oku (unwrap gerekebilir).
17 std::vector<float> pmDemod(const std::vector<cf32>& iq, float kp) {
```

```

18     std::vector<float> msg(iq.size());
19     float prevPhase = 0.0f;
20     for (size_t n = 0; n < iq.size(); ++n) {
21         float phase = std::arg(iq[n]);
22         // Basit unwrap: 2*pi sicramalarını düzelt
23         while (phase - prevPhase > M_PI) phase -= 2.0f * M_PI;
24         while (phase - prevPhase < -M_PI) phase += 2.0f * M_PI;
25         prevPhase = phase;
26         msg[n] = phase / kp;
27     }
28     return msg;
29 }

```

18. Sayısal Modülasyonlar: ASK, FSK, PSK, QAM

Şimdiye kadar analog mesajları taşıdık. Sayısal haberleşmede ise bit dizileri taşırız. Bitleri **sembollere** eşleyip her sembolü karmaşık düzlemde bir noktaya (**takım yıldızı / constellation**) koyarız.

Şema	Neyi oynatır	Sembol başına bit
ASK / OOK	genlik (var/yok)	1
BPSK	faz (0 / π)	1
QPSK	faz (4 durum)	2
FSK	frekans (iki ton)	1
16-QAM	genlik + faz (16 nokta)	4

18.1 BPSK ve QPSK: Faz Kaydırmalı Anahtarlama

BPSK, biti taşıyıcının fazına koyar: bit 0 $\rightarrow 0^\circ$, bit 1 $\rightarrow 180^\circ$. QPSK ise her sembolde 2 bit taşır (4 faz durumu). Karmaşık bant temelinde bu, sembol noktasını doğrudan üretmek kadar kolaydır:

```

1  #include <complex>
2  #include <vector>
3  #include <cstdint>
4
5  using cf32 = std::complex<float>;
6
7  // QPSK: 2 biti bir karmaşık sembole esle (Gray kodlu).
8  // 00 -> ( 1, 1), 01 -> (-1, 1), 11 -> (-1,-1), 10 -> ( 1,-1), 1/sqrt(2) ölçekli
9  cf32 qpskMap(uint8_t twoBits) {
10     const float s = 0.70710678f; // 1/sqrt(2), birim guce normalize
11     switch (twoBits & 0x3) {
12         case 0b00: return { s,  s};
13         case 0b01: return {-s,  s};
14         case 0b11: return {-s, -s};
15         case 0b10: return { s, -s};
16     }
17     return {0, 0};
18 }
19
20 // Bit dizisini QPSK sembol dizisine çevir.

```

```

21 std::vector<cf32> qpskModulate(const std::vector<uint8_t>& bits) {
22     std::vector<cf32> syms;
23     for (size_t i = 0; i + 1 < bits.size(); i += 2) {
24         uint8_t sym = (bits[i] << 1) | bits[i+1];
25         syms.push_back(qpskMap(sym));
26     }
27     return syms;
28 }
29
30 // QPSK karar (demod): en yakın takım yıldızı noktasına yuvarla.
31 uint8_t qpskDemap(cf32 r) {
32     uint8_t hi = (r.real() < 0) ? 1 : 0;    // I isareti -> ust bit
33     uint8_t lo = (r.imag() < 0) ? 1 : 0;    // Q isareti -> alt bit
34     // Gray esleme geri cozumu
35     uint8_t b0 = hi;
36     uint8_t b1 = hi ^ lo;
37     return (b0 << 1) | b1;
38 }

```

18.2 FSK: Frekans Kaydırmalı Anahtarlama

FSK, her biti farklı bir tonla gönderir. Aslında sayısal bir FM'dir. Modülatörü FM'inkiyle aynı mantıkta, demodülasyonu ise ya iki band-geçiren filtrenin enerjisini karşılaştırarak (koherent olmayan) ya da doğrudan FM diskriminatörüyle yapılır:

```

1  #include <vector>
2  #include <cmath>
3
4  // İkili FSK modulator. bit 0 -> f0, bit 1 -> f1. sps: sembol basına örnek.
5  std::vector<float> fskModulate(const std::vector<uint8_t>& bits,
6                                double f0, double f1, double fs, size_t sps) {
7
8      std::vector<float> out;
9      double phase = 0.0;
10     for (uint8_t b : bits) {
11         double f = b ? f1 : f0;
12         for (size_t k = 0; k < sps; ++k) {
13             phase += 2.0 * M_PI * f / fs;    // faz surekliligi korunur (CPFSK)
14             out.push_back(static_cast<float>(std::cos(phase)));
15         }
16     }
17     return out;
18 }
19
20 // Noncoherent FSK demod: iki tonun enerjisini karsilastir (Goertzel benzeri).
21 std::vector<uint8_t> fskDemod(const std::vector<float>& rf,
22                               double f0, double f1, double fs, size_t sps) {
23     std::vector<uint8_t> bits;
24     for (size_t i = 0; i + sps <= rf.size(); i += sps) {
25         double i0=0, q0=0, i1=0, q1=0;
26         for (size_t k = 0; k < sps; ++k) {
27             double t = (i + k) / fs, x = rf[i + k];
28             i0 += x*std::cos(2*M_PI*f0*t);    q0 += x*std::sin(2*M_PI*f0*t);
29             i1 += x*std::cos(2*M_PI*f1*t);    q1 += x*std::sin(2*M_PI*f1*t);
30         }
31         double e0 = i0*i0 + q0*q0, e1 = i1*i1 + q1*q1;
32         bits.push_back(e1 > e0 ? 1 : 0);    // hangi ton daha guclu?
33     }
34     return bits;
35 }

```

34 }

18.3 QAM: Genlik + Faz Birlikte

QAM (Quadrature Amplitude Modulation), hem genliği hem fazı kullanarak düzlemde bir ızgara oluşturur. 16-QAM her sembolde 4 bit taşır; WiFi ve DVB gibi yüksek hızlı sistemlerin temelidir:

```

1  #include <complex>
2  #include <vector>
3  #include <cstdint>
4
5  using cf32 = std::complex<float>;
6
7  // 16-QAM: 4 biti bir noktaya esle. I ve Q ayri ayri {-3,-1,+1,+3}.
8  cf32 qam16Map(uint8_t nibble) {
9      auto level = [](uint8_t twoBits) -> float {
10         switch (twoBits & 0x3) {           // Gray: 00->-3 01->-1 11->+1 10->+3
11             case 0b00: return -3.0f; case 0b01: return -1.0f;
12             case 0b11: return  1.0f; case 0b10: return  3.0f;
13         } return 0;
14     };
15     float I = level((nibble >> 2) & 0x3);
16     float Q = level(nibble & 0x3);
17     return cf32(I, Q) / 3.16227766f;      // ortalama guce normalize (~sqrt(10))
18 }
19
20 // En yakin komsu karariyla 16-QAM demod (basit dilim/slicer).
21 uint8_t qam16Demap(cf32 r) {
22     r *= 3.16227766f;                      // olcegi geri al
23     auto slice = [](float v) -> uint8_t {
24         if (v < -2) return 0b00; if (v < 0) return 0b01;
25         if (v < 2)  return 0b11; return 0b10;
26     };
27     return (slice(r.real()) << 2) | slice(r.imag());
28 }
```

Bilgi

Takım yıldızında noktalar birbirine ne kadar yakınsa, gürültüde karışma olasılığı o kadar artar. Bu yüzden yüksek dereceli QAM (64, 256, 1024-QAM) daha fazla bit/sembol taşır ama daha yüksek sinyal/gürültü oranı (SNR) gerektirir. Modern sistemler kanal kalitesine göre şemayı uyarlar (adaptive modulation): sinyal iyiysen 256-QAM, kötüysen QPSK'ye düşer.

19. Darbe Şekillendirme (Pulse Shaping)

Sembolleri kutu darbe olarak göndermek, spektrumda geniş yan loblar üretir ve komşu kanallara taşar. Ayrıca alıcıda **semboller arası girişim** (ISI) oluşur. Çözüm, sembolleri yumuşak bir darbeyle şekillendirmektir. Standart seçim **kök yükseltilmiş kosinüs** (Root-Raised-Cosine, RRC) filtresidir — yarısı vericide, yarısı alıcıda olacak şekilde bölünür:

```

1  #include <vector>
2  #include <cmath>
3
4  // RRC filtre katsayıları. beta: rolloff (0..1), sps: sembol basına örnek.
5  std::vector<float> rrcTaps(double beta, size_t sps, size_t numSymbols) {
```



```

6   const size_t N = sps * numSymbols + 1;    // tek uzunluk, simetrik
7   std::vector<float> h(N);
8   const double mid = (N - 1) / 2.0;
9   for (size_t i = 0; i < N; ++i) {
10      double t = (i - mid) / sps;             // sembol cinsinden zaman
11      double val;
12      if (std::fabs(t) < 1e-8) {
13         val = 1.0 - beta + 4.0 * beta / M_PI;
14      } else if (std::fabs(std::fabs(4*beta*t) - 1.0) < 1e-8) {
15         // Tekillik noktası (t = 1/(4*beta))
16         double a = (1 + 2/M_PI) * std::sin(M_PI/(4*beta));
17         double b = (1 - 2/M_PI) * std::cos(M_PI/(4*beta));
18         val = beta / std::sqrt(2.0) * (a + b);
19      } else {
20         double num = std::sin(M_PI*t*(1-beta))
21             + 4*beta*t*std::cos(M_PI*t*(1+beta));
22         double den = M_PI*t*(1 - (4*beta*t)*(4*beta*t));
23         val = num / den;
24      }
25      h[i] = static_cast<float>(val);
26  }
27  // Enerjiyi normalize et
28  double e = 0; for (float c : h) e += c*c;
29  for (float& c : h) c /= std::sqrt(e);
30  return h;
31 }

```

İpucu

Vericide sembollerini sps kat upsampli edip (araya sıfır koyup) RRC ile filtrelersiniz. Alıcıda aynı RRC ile eşleşmiş filtreleme (matched filter) yaparsınız. İki RRC'nin kaskadı bir “yükseltilmiş kosinüs” verir ki bu Nyquist ISI kriterini sağlar: sembol anlarında komşu sembollerin katkısı tam sıfırdır.

20. Taşıyıcı ve Zamanlama Kurtarma: PLL, Costas, Timing

Gerçek bir alıcıda vericinin taşıyıcı frekansı/fazı ve sembol zamanlaması tam bilinmez. Bunları işaretin kendisinden kestirmek gerekir. Bu senkronizasyon döngüleri, çalışan bir modem en zor ve en kritik parçasıdır.

20.1 Faz Kilitlemeli Döngü (PLL)

Bir PLL, yerel bir osilatörü gelen işaretin fazına kilitler. Faz dedektörü, döngü filtresi ve NCO'dan oluşur — her demodülatörün kalbindeki geri besleme döngüsü:

```

1  #include <complex>
2  #include <cmath>
3
4  using cf32 = std::complex<float>;
5
6  // Karmaşık bir tona kilitlenen basit ikinci dereceden PLL.
7  class Pll {
8      double phase_ = 0.0, freq_ = 0.0;
9      double alpha_, beta_;           // dongu kazancları
10 public:
11     // loopBw: dongu bant genisligi (ornekleme hizina gore normalize, ~0.01)

```

```

12  explicit Pll(double loopBw) {
13      double damping = 0.707;
14      double denom = 1 + 2*damping*loopBw + loopBw*loopBw;
15      alpha_ = 4*damping*loopBw / denom;    // orantili kazanc
16      beta_ = 4*loopBw*loopBw / denom;    // integral kazanci
17  }
18
19  // Girdiyi taşıyıcıdan arındır ve fazi kilitle. Dondurduğu: düzeltilmiş IQ.
20  cf32 process(cf32 in) {
21      cf32 nco(std::cos(-phase_), std::sin(-phase_));
22      cf32 out = in * nco;                  // yerel osilator ile carp
23      double err = std::arg(out);          // faz hatası (dedektor)
24      freq_ += beta_ * err;                // frekansı güncelle (integral)
25      phase_ += freq_ + alpha_ * err;      // fazi güncelle (orantili)
26      if (phase_ > M_PI) phase_ -= 2*M_PI;
27      if (phase_ < -M_PI) phase_ += 2*M_PI;
28      return out;
29  }
30  double frequency() const { return freq_; }
31 };

```

20.2 Costas Döngüsü: PSK için Taşıyıcı Kurtarma

PSK işaretlerinde taşıyıcı bastırılmıştır (ayrı bir taşıyıcı tepesi yok), bu yüzden düz PLL kilitlenemez. **Costas döngüsü**, faz hatasını modülasyona duyarlı biçimde üretir. BPSK için hata $I \cdot Q$, QPSK için ise karar-yönlü bir formdur:

```

1  #include <complex>
2  #include <cmath>
3
4  using cf32 = std::complex<float>;
5
6  // BPSK/QPSK için Costas dongusu. Taşıyıcı faz/frekans ofsetini düzeltir.
7  class CostasLoop {
8      double phase_ = 0.0, freq_ = 0.0, alpha_, beta_;
9      bool qpsk_;
10 public:
11     CostasLoop(double loopBw, bool qpsk = false) : qpsk_(qpsk) {
12         double d = 0.707;
13         double denom = 1 + 2*d*loopBw + loopBw*loopBw;
14         alpha_ = 4*d*loopBw / denom;
15         beta_ = 4*loopBw*loopBw / denom;
16     }
17
18     cf32 process(cf32 in) {
19         cf32 nco(std::cos(-phase_), std::sin(-phase_));
20         cf32 y = in * nco;                  // fazi düzeltilmiş örnek
21
22         // Faz hatası: BPSK -> I*Q, QPSK -> karar yönlü
23         double err;
24         if (!qpsk_) {
25             err = y.real() * y.imag();
26         } else {
27             float di = (y.real() > 0) ? 1.f : -1.f;    // I kararı
28             float dq = (y.imag() > 0) ? 1.f : -1.f;    // Q kararı
29             err = dq * y.real() - di * y.imag();
30         }
31         freq_ += beta_ * err;

```

```

32     phase_ += freq_ + alpha_ * err;
33     if (phase_ > M_PI) phase_ -= 2*M_PI;
34     if (phase_ < -M_PI) phase_ += 2*M_PI;
35     return y;
36 }
37 };

```

20.3 Sembol Zamanlama Kurtarma (Gardner)

Alıcı, her sembolü tam ortasından örneklemelidir; yoksa ISI ve hatalar artar. **Gardner zamanlama dedektörü** örnekleme anını ayarlar ve simge hızının tam bilinmesini gerektirmez (karar-öncesi çalışır):

```

1  #include <complex>
2  #include <vector>
3
4  using cf32 = std::complex<float>;
5
6  // Gardner TED ile sembol zamanlama kurtarma (sps ornek/sembol, tipik 2+).
7  // Basitleştirilmiş: her sembol için orta ve tam noktaları kullanır.
8  std::vector<cf32> gardnerSync(const std::vector<cf32>& x, double sps,
9                               double loopGain = 0.01) {
10     std::vector<cf32> syms;
11     double mu = 0.0; // kesirli ornek konumu
12     double idx = 0.0;
13     cf32 prevSym(0,0), prevMid(0,0);
14     while (idx + sps < x.size() - 1) {
15         size_t i = static_cast<size_t>(idx);
16         cf32 cur = x[i]; // (pratikte lineer/polifaz interp.)
17         cf32 mid = x[static_cast<size_t>(idx - sps/2)];
18         // Gardner hatasi: orta ornek, iki sembol gecisini "yakalar"
19         double e = (mid.real() * (cur.real() - prevSym.real())
20                    + mid.imag() * (cur.imag() - prevSym.imag()));
21         mu -= loopGain * e; // zamanlamayi ayarla
22         syms.push_back(cur);
23         prevSym = cur;
24         idx += sps + mu; // bir sonraki sembole ilerle
25         mu *= 0.9; // hatayi sonumle
26     }
27     return syms;
28 }

```

Dikkat

Bu senkronizasyon döngüleri pedagojik olarak sadeleştirilmiştir; kesirli gecikme için gerçek bir interpolatör (Farrow/polifaz), döngü kazançları için dikkatli ayar ve kilitlenme tespiti gerçek sistemlerde şarttır. Yine de temel geri besleme yapısı — *hata üret, filtrele, düzelt* — her birinde aynıdır; bu iskeleti kavramak yeter.

KISIM 6

Uçtan Uca: Tam Bir FM Radyo Alıcısı

21. IQ Dosya Formatları ve RTL-SDR

Tüm parçaları bir araya getirip gerçek bir alıcı kuralım. Girdimiz bir SDR'dan gelen ham IQ akışıdır. En yaygın formatlar:

Format	Açıklama
cu8 / u8	RTL-SDR ham çıktısı: I,Q,I,Q... her biri işaretli bayt (0–255).
cs8	işaretli 8-bit (HackRF).
cs16	işaretli 16-bit (Airspy, SDRplay). Yüksek dinamik aralık.
cf32	32-bit float, $[-1, 1]$. GNU Radio'nun iç formatı.

21.1 RTL-SDR Baytlarını Karmaşık Bant Temeline Çevirme

```

1 #include <complex>
2 #include <vector>
3 #include <cstdint>
4 #include <fstream>
5
6 using cf32 = std::complex<float>;
7
8 // RTL-SDR cu8 baytlarını normalize karmaşık ornege çevir.
9 // 0..255 -> [-1, 1], orta nokta 127.5.
10 std::vector<cf32> loadRtlSdrIq(const std::string& path) {
11     std::ifstream f(path, std::ios::binary);
12     std::vector<uint8_t> raw((std::istreambuf_iterator<char>(f)),
13                             std::istreambuf_iterator<char>());
14     std::vector<cf32> iq(raw.size() / 2);
15     for (size_t i = 0; i < iq.size(); ++i) {
16         float I = (raw[2*i] - 127.5f) / 127.5f;
17         float Q = (raw[2*i + 1] - 127.5f) / 127.5f;
18         iq[i] = cf32(I, Q);
19     }
20     return iq;
21 }
```

İpucu

RTL-SDR'dan komut satırıyla ham IQ kaydetmek için: `rtl_sdr -f 100000000 -s 2400000 -g 40 kayıt.iq` — bu, 100 MHz merkez frekansta, 2.4 MHz örnekleme hızında, 40 dB kazançla `cu8` formatında bir dosya üretir. Yukarıdaki fonksiyon tam da bu dosyayı okur.

22. Tam WBFM Alıcı Hattı

Şimdi geniş bant FM (WBFM, yani normal FM radyo) için uçtan uca sinyal işleme hattını kuralım. Adımlar şunlar:

1. **Tune:** İstenen istasyonu merkeze (DC) kaydır (NCO mikser).
2. **Kanal filtresi + desimasyon:** 2.4 MHz'i ~240 kHz'e indir (FM kanalı ± 100 kHz civarı).
3. **FM demod:** Polar diskriminatörle sesi çek.
4. **Ses desimasyonu:** 240 kHz'i 48 kHz'e indir.
5. **De-emphasis:** 50/75 μ s RC filtresi.
6. **WAV'a yaz** veya ses kartına gönder.

```

1  #include <complex>
2  #include <vector>
3  #include <cmath>
4
5  using cf32 = std::complex<float>;
6
7  // Önceki bölümlerdeki bileşenleri varsayar:
8  // Nco, ComplexFir, FirFilter, FmDemod, Deemphasis, designLowpass, writeWav
9
10 class WbfmReceiver {
11     double sdrRate_;           // orneğin 2.4e6
12     double channelRate_;       // orneğin 240e3 (M1 = 10 desimasyon)
13     double audioRate_;         // 48e3 (M2 = 5 desimasyon)
14     double tuneOffset_;        // istasyonun merkezden offseti (Hz)
15
16     Nco lo_;
17     ComplexFir channelFilter_;
18     FmDemod demod_;
19     FirFilter audioFilter_;
20     Deemphasis deemph_;
21     size_t decim1_, decim2_;
22 public:
23     WbfmReceiver(double sdrRate, double tuneOffsetHz)
24         : sdrRate_(sdrRate), channelRate_(sdrRate/10.0), audioRate_(48000.0),
25           tuneOffset_(tuneOffsetHz),
26           lo_(-tuneOffsetHz, sdrRate), // asagi kaydir
27           channelFilter_(designLowpass(100e3, sdrRate, 65)), // FM kanal BW
28           demod_(channelRate_, 75e3), // sapma 75 kHz
29           audioFilter_(designLowpass(15e3, channelRate_, 65)), // ses BW 15 kHz
30           deemph_(50e-6, audioRate_), // Avrupa 50 us
31           decim1_(10), decim2_(5) {}
32
33     // Ham IQ blogunu isle -> [-1,1] ses ornekleri (48 kHz mono).

```

```

34     std::vector<float> process(const std::vector<cf32>& iq) {
35         std::vector<float> audio;
36         size_t ph1 = 0, ph2 = 0;
37         for (cf32 raw : iq) {
38             // 1) Tune: istasyonu DC'ye getir
39             cf32 shifted = raw * lo_.next();
40             // 2) Kanal filtresi (her örnek), desimasyon 1
41             cf32 filt = channelFilter_.process(shifted);
42             if (ph1++ % decim1_ != 0) continue;
43             // 3) FM demodulasyon (artık channelRate_ hızında)
44             float d = demod_.process(filt);
45             // 4) Ses filtresi + desimasyon 2
46             float a = audioFilter_.process(d);
47             if (ph2++ % decim2_ != 0) continue;
48             // 5) De-emphasis
49             audio.push_back(deemph_.process(a));
50         }
51         return audio;
52     }
53 };
54
55 int main() {
56     // 100.0 MHz'te kayıt yapıldı; 100.3 MHz istasyonunu dinle -> +300 kHz offset
57     auto iq = loadRtlSdrIq("kayit.iq");
58     WbfmReceiver rx(2.4e6, +300e3);
59     std::vector<float> audio = rx.process(iq);
60
61     // Normalize ve WAV'a yaz
62     float peak = 1e-6f;
63     for (float s : audio) peak = std::max(peak, std::fabs(s));
64     for (float& s : audio) s *= 0.9f / peak;
65     writeWav("radyo.wav", audio, 48000, 1);
66     return 0;
67 }

```

Stereo FM

FM stereo yayınında sol+sağ (L+R) ses 0–15 kHz'te gönderilirken, sol–sağ (L–R) farkı 38 kHz'lik bir alt taşıyıcıda DSB-SC olarak modüle edilir. 19 kHz'te de bir pilot ton bulunur. Stereo çözmek için: demod çıktısından 19 kHz pilotu bir PLL ile yakala, frekansını ikiye katla (38 kHz), L–R'yi bu tonla çarparak alçağa indir, sonra $L = (L+R) + (L-R)$ ve $R = (L+R) - (L-R)$ ile iki kanalı ayır. Yukarıdaki hattı mono bıraktık; stereo, aynı IQ diskriminatör çıktısına eklenen bir DSP katmanındır.

23. Performans ve Sonraki Adımlar

23.1 Optimizasyon Notları

- **atan2'den kaçın:** FM demodda her örnekte `std::arg` çağırmak yerine yaklaşık faz-farkı formülleri veya bir CORDIC kullanın.
- **Polifaz filtreler:** Desimasyonda atacağınız örnekleri hiç hesaplamayın — M kat hızlanma.
- **SIMD:** FIR konvolüsyonu, çarpma-toplama ağırlıklıdır; AVX/NEON ile 4–8 kat hızlanır. Modern derleyiciler `-O3 -march=native` ile çoğunu otomatik yapar.

- **Blok işleme:** Örnek örnek değil, blok blok çalışın; önbellek ve dallanma tahmini çok daha verimli olur.
- **FFT tabanlı konvolüsyon:** Uzun FIR'lar için (100+ tap), overlap-add/overlap-save yöntemiyle FFT üzerinden konvolüsyon $O(N \log N)$ verir.

23.2 Üretimde Kullanılan Kütüphaneler

Bu belge her şeyi sıfırdan yazdı ki mekanizmayı anlayın. Gerçek projelerde ise olgun kütüphaneleri tercih edin:

Araç	Ne için
FFTW / KissFFT	Endüstri standardı, çok hızlı FFT (kendi FFT'niz yerine).
Liquid-DSP	C ile yazılmış kapsamlı SDR/modem kütüphanesi (filtre, modem, sync).
GNU Radio	Blok tabanlı görsel/akış SDR çatısı; prototipleme için mükemmel.
VOLK	Vektörize (SIMD) DSP çekirdekleri; GNU Radio'nun altında çalışır.
SoapySDR	Donanım-bağımsız SDR sürücü katmanı (RTL, HackRF, Airtyspy...).

23.3 Nereye Bakmalı

Buradan devam etmek için doğal yollar: **sayısal alıcı zinciri** (AGC, eşleşmiş filtre, senkron, çözücü), **kanal kodlaması** (Hamming, konvolüsyon, Viterbi, LDPC, Turbo), **OFDM** (WiFi/LTE/5G'nin çok taşıyıcılı temeli), **spektrum kestirimi** (Welch, MUSIC), ve **uyarlamalı filtreler** (LMS, RLS ile ekolayzasyon ve gürültü giderme).

Bilgi

Öğrenmenin en hızlı yolu *dinlemek* ve *görmektir*. Her demodülatör aşamasının çıktısını WAV'a yazıp dinleyin, FFT'sini alıp spektrumuna bakın. Bir FM alıcısını çalıştırıp gerçek bir istasyonu duyduğunuz an, bu belgedeki tüm parçalar — örnekleme, IQ, mikser, filtre, diskriminatör — tek bir bütün olarak yerine oturur.

KISIM 7

İleri Konular

24. Stereo FM Çözücü (Tam Uygulama)

Kısım 6'da mono FM alıcısını kurmuş, stereo'yu ise kavramsal bırakmıştık. Şimdi onu tam koda dökelim. FM stereo yayınında demodülatör çıkışı (**MPX** ya da kompozit işaret) şu katmanlardan oluşur:

Bileşen	Frekans bandı
L+R (mono uyumlu)	0–15 kHz (bant temeli)
Pilot ton	19 kHz (saf sinüs, faz referansı)
L–R (DSB-SC)	23–53 kHz, 38 kHz alt taşıyıcı etrafında
RDS (opsiyonel)	57 kHz (pilotun 3. harmoniği)

Sol ve sağ kanalı ayırmak için: (1) 19 kHz pilotu bir PLL ile yakala, (2) frekansını ikiye katlayıp 38 kHz üret, (3) L–R'yi bu tonla çarpıp bant temeline indir, (4) matris işlemiyle $L = (L+R) + (L-R)$ ve $R = (L+R) - (L-R)$ hesapla.

```

1  #include <complex>
2  #include <vector>
3  #include <cmath>
4
5  // Kompozit (MPX) FM çıkışından stereo L/R cozer.
6  // Girdi: FM demod çıkışı (Kısım 6'daki FmDemod), ~240 kHz hızında.
7  // Çikti: interleaved stereo (L,R,L,R...) 48 kHz'e desime edilmiş.
8  class StereoDecoder {
9      double mpxRate_;           // MPX örnekleme hızı (örn. 240 kHz)
10     Pll pilotPll_;              // 19 kHz pilota kilitlenir (Kısım 5)
11     Biquad pilotBpf_;           // 19 kHz bant geciren
12     FirFilter lprLpf_, lmrLpf_; // her iki kanal için 15 kHz alçak geciren
13     Deemphasis deL_, deR_;
14     size_t decim_, phase_ = 0;
15 public:
16     StereoDecoder(double mpxRate, double audioRate)
17         : mpxRate_(mpxRate),
18           pilotPll_(0.001),           // dar doğu BW
19           pilotBpf_(Biquad::bandpass(19000.0, mpxRate, 20)),
20           lprLpf_(designLowpass(15000.0, mpxRate, 65)),

```



```

21     lmrLpf_(designLowpass(15000.0, mpxRate, 65)),
22     deL_(50e-6, audioRate), deR_(50e-6, audioRate),
23     decim_(static_cast<size_t>(mpxRate / audioRate)) {}
24
25     std::vector<float> process(const std::vector<float>& mpx) {
26         std::vector<float> stereo;
27         for (float x : mpx) {
28             // 1) 19 kHz pilotu izole et ve PLL'e ver
29             float pilot = pilotBpf_.process(x);
30             // PLL'i gercek pilota kilitle; kilitli fazi al
31             cf32 locked = pilotPll_.process(cf32(pilot, 0.0f));
32             double ph = std::arg(pilotPll_.phaseState()); // 19 kHz fazi
33
34             // 2) 38 kHz alt tasiyici = pilot fazinin 2 kati
35             float sub38 = std::cos(2.0 * ph);
36
37             // 3) Kanallari ayir
38             float lpr = lprLpf_.process(x); // L+R (bant temeli)
39             float lmr = lmrLpf_.process(x * sub38 * 2.0f); // L-R (asagi mixle)
40
41             // 4) Desimasyon
42             if (phase_++ % decim_ != 0) continue;
43
44             // 5) Matris + de-emphasis
45             float L = deL_.process(lpr + lmr);
46             float R = deR_.process(lpr - lmr);
47             stereo.push_back(L);
48             stereo.push_back(R);
49         }
50         return stereo;
51     }
52 };

```

PLL'e küçük bir ekleme

Yukarıdaki kod, Kısım 5'teki P11 sınıfına anlık fazı dışarı veren bir yardımcı ekler: cf32 phaseState() const { return {std::cos(phase_), std::sin(phase_)}; } Böylece pilotun fazını okuyup ikiye katlayarak 38 kHz üretiriz. Pilot yoksa (zayıf sinyal) PLL kilitlenemez; bu durumda yalnız L+R kullanılıp otomatik olarak mono'ya düşülür — gerçek alıcıların yaptığı budur.

İpucu

Stereo çözmenin başarısı tamamen pilot PLL'in kilidine bağlıdır. 19 kHz'e ± 2 Hz'ten fazla sapma stereo ayırımı (separation) bozar. Bu yüzden döngü bant genişliğini dar (0.001 civarı) tutarız: yavaş kilitlenir ama gürültüye karşı çok kararlıdır.

25. Otomatik Kazanç Kontrolü (AGC)

Gerçek sinyallerin gücü sürekli değişir (sönümlenme/fading, mesafe, anten yönü). Demodülatörlerin çoğu *sabit genlik* bekler — özellikle QAM gibi genlik taşıyan şemalar. **AGC**, işareti hedef bir güç seviyesine sürekli ölçekleyerek bunu sağlar:

```

1 #include <complex>
2 #include <cmath>
3

```

```

4 using cf32 = std::complex<float>;
5
6 // Karmasik AGC: cikis gucunu hedef seviyeye kilitletler.
7 class Agc {
8     float gain_ = 1.0f;
9     float target_, rate_;
10 public:
11     // target: istenen cikis genligi (orn. 1.0), rate: uyum hizi (0..1, ~1e-3)
12     Agc(float target = 1.0f, float rate = 1e-3f)
13         : target_(target), rate_(rate) {}
14
15     cf32 process(cf32 x) {
16         cf32 out = x * gain_;
17         float mag = std::abs(out);
18         // Hata: hedef ile olculen arasindaki fark -> kazanci ayarla
19         gain_ += rate_ * (target_ - mag) * gain_;
20         if (gain_ < 1e-6f) gain_ = 1e-6f; // patlamayi onle
21         return out;
22     }
23     float gain() const { return gain_; }
24 };

```

Dikkat

AGC hızı (rate) kritik bir dengedir. Çok hızlıysa, işaretin kendi genlik modülasyonunu (AM/QAM bilgisi!) “düzleştirip” bilgiyi siler. Çok yavaşsa, ani sönümlenmelere yetişemez. Kural: AGC, sembol hızından çok daha yavaş ama fading’den çok daha hızlı olmalıdır.

26. Uçtan Uca Sayısal Modem: QPSK Vericiden Alıcıya

Şimdi Kısım 5’teki tüm parçaları — QPSK eşleme, RRC şekillendirme, kanal, AGC, Costas ve zamanlama — tek bir çalışan modemde birleştirelim. Bu, gerçek bir sayısal haberleşme zincirinin iskeletidir.

26.1 Verici (Modülatör)

```

1 #include <complex>
2 #include <vector>
3 #include <cstdint>
4
5 using cf32 = std::complex<float>;
6
7 // Bitler -> QPSK sembol -> upsample -> RRC sekillendirme -> baseband IQ
8 std::vector<cf32> qpskTransmit(const std::vector<uint8_t>& bits,
9                                size_t sps, double beta) {
10     // 1) Bitleri QPSK sembollere esle (Kisim 5)
11     std::vector<cf32> syms = qpskModulate(bits);
12
13     // 2) sps kat upsample: her sembol arasina sps-1 sifir koy
14     std::vector<cf32> up(syms.size() * sps, cf32(0, 0));
15     for (size_t i = 0; i < syms.size(); ++i) up[i * sps] = syms[i];
16
17     // 3) RRC filtre ile darbe sekillendir (Kisim 5’teki rrcTaps)
18     ComplexFir shaper(rrcTaps(beta, sps, 6));
19     std::vector<cf32> tx(up.size());

```

```

20     for (size_t i = 0; i < up.size(); ++i) tx[i] = shaper.process(up[i]);
21     return tx;
22 }

```

26.2 Kanal Modeli

Alıcıyı test etmek için gerçekçi bozulmalar ekleyen bir kanal yazalım: frekans ofseti, faz kayması ve Gauss gürültüsü. (std::mt19937 ile deterministik.)

```

1  #include <random>
2
3  // Kanal: frekans ofseti + faz + AWGN gürültüsü ekler.
4  std::vector<cf32> channel(const std::vector<cf32>& tx, double freqOffset,
5                          double phase, double noiseStd, uint32_t seed = 42) {
6      std::mt19937 rng(seed);
7      std::normal_distribution<float> noise(0.0f, noiseStd);
8      std::vector<cf32> rx(tx.size());
9      double ph = phase;
10     for (size_t i = 0; i < tx.size(); ++i) {
11         cf32 rot(std::cos(ph), std::sin(ph)); // taşıyıcı ofseti
12         rx[i] = tx[i] * rot + cf32(noise(rng), noise(rng));
13         ph += 2.0 * M_PI * freqOffset;
14     }
15     return rx;
16 }

```

26.3 Alıcı (Demodülatör)

Alıcı zinciri: AGC → eşleşmiş RRC filtre → zamanlama kurtarma → Costas ile taşıyıcı kurtarma → karar (slicer) → bitler.

```

1  #include <complex>
2  #include <vector>
3  #include <cstdint>
4
5  using cf32 = std::complex<float>;
6
7  std::vector<uint8_t> qpskReceive(std::vector<cf32> rx, size_t sps, double beta) {
8      // 1) AGC ile genliği normalize et
9      Agc agc(1.0f, 1e-3f);
10     for (cf32& s : rx) s = agc.process(s);
11
12     // 2) Eşleşmiş filtre (verici ile aynı RRC) -> ISI'yi minimize eder
13     ComplexFir matched(rrcTaps(beta, sps, 6));
14     std::vector<cf32> filt(rx.size());
15     for (size_t i = 0; i < rx.size(); ++i) filt[i] = matched.process(rx[i]);
16
17     // 3) Sembol zamanlama kurtarma (Kisim 5'teki Gardner) -> sembol hizina in
18     std::vector<cf32> syms = gardnerSync(filt, static_cast<double>(sps));
19
20     // 4) Costas döngüsü ile taşıyıcı faz/frekans kurtar (QPSK modu)
21     CostasLoop costas(0.01, /*qpsk=*/true);
22     std::vector<uint8_t> bits;
23     for (cf32 s : syms) {
24         cf32 corrected = costas.process(s);
25         // 5) Karar: en yakın QPSK noktası -> 2 bit (Kisim 5'teki qpskDemap)
26         uint8_t sym = qpskDemap(corrected);
27         bits.push_back((sym >> 1) & 1);

```

```

28     bits.push_back(sym & 1);
29 }
30 return bits;
31 }
32
33 // Uctan uca test: gonder -> kanaldan gecir -> al -> bit hata oranini (BER) olc.
34 #include <cstdio>
35 int main() {
36     std::vector<uint8_t> tx_bits;
37     std::mt19937 rng(1);
38     for (int i = 0; i < 2000; ++i) tx_bits.push_back(rng() & 1);
39
40     auto tx = qpskTransmit(tx_bits, /*sps=*/8, /*beta=*/0.35);
41     auto rx = channel(tx, /*freqOff=*/1e-4, /*phase=*/0.7, /*noise=*/0.1);
42     auto out = qpskReceive(rx, 8, 0.35);
43
44     // BER: senkron gecikmesi nedeniyle hizalanmis kısmi karsilastir
45     size_t errors = 0, compared = 0;
46     size_t offset = 40; // filtre/sync gecikmesi (deneysel)
47     for (size_t i = 0; i + offset < out.size() && i < tx_bits.size(); ++i) {
48         errors += (out[i + offset] != tx_bits[i]);
49         ++compared;
50     }
51     printf("BER = %.4f (%zu/%zu)\n", (double)errors/compared, errors, compared);
52     return 0;
53 }

```

Bilgi

Gerçek bir modemde ek olarak: **çerçeve senkronizasyonu** (bilinen bir önsöz/preamble dizisiyle bit hizalaması), **faz belirsizliği çözümü** (QPSK 90°'lik dört olası dönüşten hangisinde kilitlendiğini bilmez — diferansiyel kodlama veya bilinen semboller çözer) ve **kanal denkleştirme** (equalization) bulunur. Yukarıdaki iskelet bu eklemeler için sağlam bir temeldir.

27. OFDM: Çok Taşıyıcılı Modülasyon

WiFi, LTE, 5G, DVB-T ve DAB'ın temelinde **OFDM** (Orthogonal Frequency-Division Multiplexing) yatar. Fikir zarif: yüksek hızlı bir bit akışını, her biri kendi dar taşıyıcısında düşük hızla giden *binlerce* paralel alt kanala böl. Bu alt taşıyıcılar dik seçilir, öyle ki bir **IFFT** ile hepsi tek adımda üretilir — OFDM'in tüm sihri budur.

```

1  #include <complex>
2  #include <vector>
3
4  using cf64 = std::complex<double>;
5
6  // OFDM verici: her alt tasiyiciya bir QAM sembolu koy, IFFT ile zamana don,
7  // arkasina dongusel onek (cyclic prefix) ekle.
8  std::vector<cf64> ofdmModulate(const std::vector<cf64>& qamSymbols,
9                                  size_t nfft, size_t cpLen) {
10
11     std::vector<cf64> out;
12     for (size_t base = 0; base < qamSymbols.size(); base += nfft) {
13         // 1) Bir FFT-blogu doldur (alt tasiyicilar = frekans binleri)
14         std::vector<cf64> freq(nfft, cf64(0, 0));
15         for (size_t k = 0; k < nfft && base + k < qamSymbols.size(); ++k)

```

```

15         freq[k] = qamSymbols[base + k];
16
17         // 2) IFFT -> zaman alanı OFDM sembolu (Kısım 2'deki fft, invert=true)
18         fft(freq, /*invert=*/true);
19
20         // 3) Döngüsel önek: son cpLen örneği başa kopyala
21         // -> çok yollu yayılma (multipath) karşı bağışıklık sağlar
22         for (size_t i = nfft - cpLen; i < nfft; ++i) out.push_back(freq[i]);
23         for (size_t i = 0; i < nfft; ++i) out.push_back(freq[i]);
24     }
25     return out;
26 }
27
28 // OFDM alıcı: döngüsel öneki at, FFT ile alt taşıyıcıları geri çek.
29 std::vector<cf64> ofdmDemodulate(const std::vector<cf64>& rx,
30                                 size_t nfft, size_t cpLen) {
31     std::vector<cf64> symbols;
32     const size_t symLen = nfft + cpLen;
33     for (size_t base = 0; base + symLen <= rx.size(); base += symLen) {
34         // 1) Döngüsel öneki atla
35         std::vector<cf64> block(rx.begin() + base + cpLen,
36                                rx.begin() + base + cpLen + nfft);
37         // 2) FFT -> her bin bir alt taşıyıcının QAM sembolu
38         fft(block, /*invert=*/false);
39         for (cf64 s : block) symbols.push_back(s);
40     }
41     return symbols;
42 }

```

Döngüsel önek neden işe yarar?

Çok yollu bir kanalda işaretin gecikmeli kopyaları üst üste biner (semboller arası girişim). Döngüsel önek, her OFDM sembolünün kuyruğunu başına kopyalayarak lineer konvolüsyonu *dairesel* konvolüsyona çevirir. Dairesel konvolüsyon frekans alanında basit bir *çarpmaya* dönüştüğü için (konvolüsyon teoremi, Kısım 8), her alt taşıyıcı kanalı tek bir karmaşık katsayıyla düzeltilebilir — OFDM'in çok yollu kanallara karşı efsanevi dayanıklılığı buradan gelir.

28. Kanal Kodlaması: Konvolüsyonel Kod ve Viterbi

Gürültü kaçınılmaz olarak bit hatası üretir. **İleri hata düzeltme** (FEC), gönderilen veriye fazlalık ekleyerek alıcının hataları *düzeltilmesini* sağlar. En klasik yöntem, **konvolüsyonel kod + Viterbi çözücüdür** (uydu, WiFi, GSM'de kullanılır).

28.1 Konvolüsyonel Kodlayıcı

Kısıt uzunluğu $K = 3$, oran $1/2$ bir kodlayıcı: her giriş biti için, kaydırma yazmacındaki son 3 bittten iki çıkış biti üretilir (üreteç çokterlileri $G_1 = 111_2 = 7$, $G_2 = 101_2 = 5$).

```

1  #include <vector>
2  #include <cstdint>
3
4  // Oran 1/2, K=3 konvolusyonel kodlayıcı. Her giriş biti -> 2 çıkış biti.
5  std::vector<uint8_t> convEncode(const std::vector<uint8_t>& bits) {
6      uint8_t state = 0; // 2 bitlik kaydırma yazmacı
7      std::vector<uint8_t> out;
8      for (uint8_t b : bits) {

```

```

9      uint8_t reg = ((b & 1) << 2) | state; // 3 bit: [yeni, s1, s0]
10     // Parite: G1=7 (111), G2=5 (101) -> XOR ile bit sayımı
11     uint8_t o1 = __builtin_parity(reg & 0b111);
12     uint8_t o2 = __builtin_parity(reg & 0b101);
13     out.push_back(o1);
14     out.push_back(o2);
15     state = reg & 0b11; // yeni durum (son 2 bit)
16 }
17 return out;
18 }

```

28.2 Viterbi Çözücü

Viterbi, tüm olası durum dizileri (trellis) arasından, alınan bitlere en yakın yolu *en olası gönderilen dizi* olarak bulur — dinamik programlama ile verimli biçimde:

```

1  #include <vector>
2  #include <cstdint>
3  #include <limits>
4  #include <algorithm>
5
6  // Oran 1/2, K=3 için sert-karar (hard-decision) Viterbi çözümü.
7  std::vector<uint8_t> viterbiDecode(const std::vector<uint8_t>& recv) {
8      const int NS = 4; // durum sayısı (2^(K-1))
9      const int INF = std::numeric_limits<int>::max() / 2;
10     size_t steps = recv.size() / 2;
11
12     // Her adımda: her durumun en düşük maliyeti ve gelinen yol
13     std::vector<std::vector<int>>> cost(steps + 1, std::vector<int>(NS, INF));
14     std::vector<std::vector<int>>> prev(steps + 1, std::vector<int>(NS, -1));
15     std::vector<std::vector<uint8_t>>> inBit(steps + 1, std::vector<uint8_t>(NS));
16     cost[0][0] = 0; // başlangıç durumu 0
17
18     // İleri geçiş: trellis'i doldur (Hamming mesafesi = dal metriği)
19     for (size_t t = 0; t < steps; ++t) {
20         uint8_t r1 = recv[2*t], r2 = recv[2*t + 1];
21         for (int s = 0; s < NS; ++s) {
22             if (cost[t][s] == INF) continue;
23             for (uint8_t b = 0; b < 2; ++b) { // olası giriş biti
24                 uint8_t reg = (b << 2) | s;
25                 uint8_t e1 = __builtin_parity(reg & 0b111);
26                 uint8_t e2 = __builtin_parity(reg & 0b101);
27                 int metric = (e1 != r1) + (e2 != r2); // Hamming mesafesi
28                 int ns = reg & 0b11;
29                 int nc = cost[t][s] + metric;
30                 if (nc < cost[t+1][ns]) {
31                     cost[t+1][ns] = nc;
32                     prev[t+1][ns] = s;
33                     inBit[t+1][ns] = b;
34                 }
35             }
36         }
37     }
38
39     // En düşük maliyetli son durumu bul, geriye iz sürerek bitleri kurtar
40     int best = 0;
41     for (int s = 1; s < NS; ++s) if (cost[steps][s] < cost[steps][best]) best = s;
42
43     std::vector<uint8_t> decoded(steps);

```

```

44     int s = best;
45     for (size_t t = steps; t > 0; --t) {
46         decoded[t-1] = inBit[t][s];
47         s = prev[t][s];
48     }
49     return decoded;
50 }

```

Hata düzeltmeyi görmek

Kodla, birkaç biti bilerek boz, çöz — Viterbi onları düzeltir:

```

1  #include <cstdio>
2  int main() {
3      std::vector<uint8_t> msg = {1,0,1,1,0,0,1,0,1,1};
4      auto enc = convEncode(msg);
5      enc[3] ^= 1; enc[10] ^= 1;           // 2 kanal hatasi enjekte et
6      auto dec = viterbiDecode(enc);
7
8      int errs = 0;
9      for (size_t i = 0; i < msg.size(); ++i) errs += (dec[i] != msg[i]);
10     printf("Kanal hatasi vardi ama cozum sonrasi hata = %d\n", errs); // 0
11     return 0;
12 }

```

İpucu

Yukarıdaki “sert karar” Viterbi, bitleri 0/1’e yuvarlanmış alır. **Yumuşak karar** (soft-decision) Viterbi ise demodülatörün ham güven değerlerini (örneğin Costas çıkışındaki mesafeleri) kullanır ve ~2 dB daha iyi başarımlar verir. Modern sistemler bunu bir adım öteye taşıyıp **Turbo** ve **LDPC** kodları kullanır — Shannon sınırına çok yaklaşırlar.

KISIM 8

Projeler

Bu kısım, önceki bölümlerdeki yapı taşlarını birleştiren dört *tam, derlenip çalışan* projeyi adım adım kurar. Her proje kendi başına bir dosyadır; bu belgedeki yardımcılar (WAV G/Ç, FFT, filtreler) yeniden kullanır. En sonda hepsini derleyen bir CMake dosyası var.

29. Proje 1 — DTMF (Telefon Tuşu) Çözücü

Telefon tuşlarının çıkardığı “tuu-tuu” sesleri **DTMF**’dir (Dual-Tone Multi-Frequency): her tuş, biri düşük biri yüksek gruptan iki sinüsün toplamıdır. Örneğin “5” tuşu 770 Hz + 1336 Hz’tir. Çözmek için, bu 8 frekansın her birinin enerjisini ölçüp en güçlü düşük+yüksek çifti buluruz. Tek bir frekansın enerjisini tüm FFT’yi hesaplamadan bulan **Goertzel algoritması** tam bu iş içindir.

```

1  #include <vector>
2  #include <cmath>
3  #include <cstdio>
4  #include <array>
5
6  // Goertzel: tek bir hedef frekansın enerjisini O(N)'de, FFT'siz olcer.
7  double goertzel(const std::vector<float>& x, double targetHz, double fs) {
8      const size_t N = x.size();
9      double k = 0.5 + (N * targetHz) / fs;
10     double w = 2.0 * M_PI * static_cast<int>(k) / N;
11     double coeff = 2.0 * std::cos(w);
12     double s0 = 0, s1 = 0, s2 = 0;
13     for (size_t n = 0; n < N; ++n) {
14         s0 = x[n] + coeff * s1 - s2;
15         s2 = s1; s1 = s0;
16     }
17     return s1*s1 + s2*s2 - coeff*s1*s2;    // enerji (buyukluk karesi)
18 }
19
20 // Bir ses blogundan DTMF tusunu coz. Tanimsizsa 0 doner.
21 char decodeDtmf(const std::vector<float>& block, double fs) {
22     static const double lowF[4] = {697, 770, 852, 941};
23     static const double highF[4] = {1209, 1336, 1477, 1633};
24     static const char keys[4][4] = {
25         {'1','2','3','A'}, {'4','5','6','B'},
26         {'7','8','9','C'}, {'*','0','#','D'}
27     };
28     // En guclu dusuk ve yuksek grup frekansini bul
29     int lo = 0, hi = 0; double bestLo = 0, bestHi = 0;
30     for (int i = 0; i < 4; ++i) {
31         double eL = goertzel(block, lowF[i], fs);

```



```

32     double eH = goertzel(block, highF[i], fs);
33     if (eL > bestLo) { bestLo = eL; lo = i; }
34     if (eH > bestHi) { bestHi = eH; hi = i; }
35 }
36 // Esik: iki ton da yeterince guclu mu? (basit sessizlik reddi)
37 double avg = 0; for (float v : block) avg += v*v;
38 avg /= block.size();
39 if (bestLo < avg * block.size() * 0.5 ||
40     bestHi < avg * block.size() * 0.5) return 0;
41 return keys[lo][hi];
42 }
43
44 int main() {
45     const double fs = 8000.0;    // telefon standardi
46     // Test: "5" tusunu sentezle (770 + 1336 Hz), sonra coz
47     std::vector<float> tone;
48     for (int n = 0; n < 400; ++n) {    // 50 ms blok
49         double t = n / fs;
50         tone.push_back(0.5f*std::sin(2*M_PI*770*t)
51                       + 0.5f*std::sin(2*M_PI*1336*t));
52     }
53     printf("Cozulen tus: %c\n", decodeDtmf(tone, fs));    // '5'
54     return 0;
55 }

```

İpucu

Gerçek bir telefon kaydını çözmek için sesi 40–50 ms’lik bloklara bölüp her bloğu decodeDtmf’e verin; ardışık aynı tuşları teke indirin (debounce). Bir .wav kaydı okumak için Kısım 1’deki WAV okuyucuyu ekleyin — gerçek bir çağrı merkezinin IVR sistemi tam olarak böyle çalışır.

30. Proje 2 — Ses Üzerinden Veri İletimi (FSK Modem)

Bu proje, iki bilgisayarın hoparlör ve mikrofonla veri alışverişi yapmasını sağlayan minik bir modem kurar — “ggwave” ve eski telefon modemlerinin prensibi. Metni bitlere çevirip FSK (Kısım 5) ile ses tonlarına, sonra tekrar metne dönüştürürüz. Senkronizasyon için başa bilinen bir önsöz (preamble) tonu koyarız.

```

1  #include <vector>
2  #include <string>
3  #include <cstdint>
4  #include <cmath>
5
6  // --- VERICI: metin -> ses ornekleri ---
7  std::vector<float> encodeText(const std::string& text, double fs) {
8      const double f0 = 1200.0, f1 = 2200.0;    // Bell 202 benzeri tonlar
9      const size_t sps = static_cast<size_t>(fs / 300.0);    // 300 baud
10     // Metni bit dizisine ac (her karakter 8 bit, MSB once)
11     std::vector<uint8_t> bits;
12     // Onsoz: senkron icin 16 adet 1-0 gecisi
13     for (int i = 0; i < 16; ++i) { bits.push_back(1); bits.push_back(0); }
14     for (char c : text)
15         for (int b = 7; b >= 0; --b) bits.push_back((c >> b) & 1);
16
17     // FSK modulasyon (surekli faz)

```

```

18     std::vector<float> out;
19     double phase = 0.0;
20     for (uint8_t bit : bits) {
21         double f = bit ? f1 : f0;
22         for (size_t k = 0; k < sps; ++k) {
23             phase += 2.0 * M_PI * f / fs;
24             out.push_back(0.6f * static_cast<float>(std::sin(phase)));
25         }
26     }
27     return out;
28 }
29
30 // --- ALICI: ses ornekleri -> metin ---
31 std::string decodeText(const std::vector<float>& audio, double fs) {
32     const double f0 = 1200.0, f1 = 2200.0;
33     const size_t sps = static_cast<size_t>(fs / 300.0);
34
35     // Her sembol icin iki tonun enerjisinin Goertzel ile karsilastir
36     std::vector<uint8_t> bits;
37     for (size_t i = 0; i + sps <= audio.size(); i += sps) {
38         std::vector<float> block(audio.begin() + i, audio.begin() + i + sps);
39         double e0 = goertzel(block, f0, fs); // Proje 1'deki goertzel
40         double e1 = goertzel(block, f1, fs);
41         bits.push_back(e1 > e0 ? 1 : 0);
42     }
43
44     // Onsozden sonraki veriyi bul: 32 bitlik gecis desenini atla
45     size_t start = 32;
46     // Bitleri baytlara topla
47     std::string text;
48     for (size_t i = start; i + 8 <= bits.size(); i += 8) {
49         uint8_t c = 0;
50         for (int b = 0; b < 8; ++b) c = (c << 1) | bits[i + b];
51         if (c == 0) break;
52         text.push_back(static_cast<char>(c));
53     }
54     return text;
55 }
56
57 #include <cstdio>
58 int main() {
59     const double fs = 44100.0;
60     std::string msg = "MERHABA DSP";
61     auto audio = encodeText(msg, fs);
62     writeWav("modem.wav", audio, 44100, 1); // Kisim 1'deki writeWav
63     std::string got = decodeText(audio, fs);
64     printf("Gonderilen: %s\nAlinan : %s\n", msg.c_str(), got.c_str());
65     return 0;
66 }

```

Bilgi

modem.wav'ı bir bilgisayarda çalıp diğerinde mikrofona kaydederek gerçek havadan iletim yapabilirsiniz. Gerçek ortamda çalışması için ekstra: bit hatalarına karşı Kısım 7'deki Viterbi kodu, ve önsözden sonra tam bit hizası için bir kare senkronizasyon damgası (örn. 0x7E bayrağı) ekleyin.

31. Proje 3 — Terminal Spektrum Analizörü

Bir WAV dosyasını okuyup, kayan pencerelerle FFT alarak terminalde ASCII bir spektrum (mini “şelale”) çizen bir araç. Bir sinyalin frekans içeriğini gözle görmenin en hızlı yolu — hata ayıklarken paha biçilmezdir.

```

1  #include <vector>
2  #include <complex>
3  #include <cstdio>
4  #include <cmath>
5  #include <algorithm>
6
7  using cf64 = std::complex<double>;
8
9  // [-1,1] mono örnekleri al, ardisik FFT pencerelerini ASCII bar olarak çiz.
10 void spectrumAnalyzer(const std::vector<float>& x, double fs) {
11     const size_t N = 512;                // FFT boyu
12     auto win = hannWindow(N);            // Kisim 2'deki pencere
13     const char* ramp = " .:-=+*#%@";    // yogunluk karakterleri (10 kademe)
14     const int nBars = 64;                // gosterilecek frekans binleri
15
16     for (size_t base = 0; base + N <= x.size(); base += N) {
17         std::vector<cf64> buf(N);
18         for (size_t n = 0; n < N; ++n) buf[n] = x[base + n] * win[n];
19         fft(buf);                         // Kisim 2'deki FFT
20
21         // Ilk nBars bin'i dB'ye cevir ve karaktere esle
22         printf("|");
23         for (int k = 0; k < nBars; ++k) {
24             double mag = std::abs(buf[k]) / N;
25             double db = 20.0 * std::log10(mag + 1e-9);
26             // -80..0 dB araligini 0..9 karakter indeksine esle
27             int level = static_cast<int>((db + 80.0) / 80.0 * 9.0);
28             level = std::max(0, std::min(9, level));
29             putchar(ramp[level]);
30         }
31         double tSec = base / fs;
32         printf("| t=%.2fs\n", tSec);
33     }
34     // Ust eksen: her karakter fs/N * (nBars) Hz'e kadar
35     printf("0 Hz %*s %.0f Hz\n", nBars - 10, "->", nBars * fs / N);
36 }
37
38 int main() {
39     const double fs = 8000.0;
40     // Test sinyali: zamanla yukselen bir supurme (chirp) 200 -> 3000 Hz
41     std::vector<float> x;
42     double phase = 0.0;
43     for (int n = 0; n < 8000; ++n) {
44         double f = 200.0 + 2800.0 * n / 8000.0;
45         phase += 2.0 * M_PI * f / fs;
46         x.push_back(0.8f * static_cast<float>(std::sin(phase)));
47     }
48     spectrumAnalyzer(x, fs); // ekranda kosegen bir "cizgi" gorursunuz
49     return 0;
50 }

```

İpucu

Bu araç ürettiğiniz her sinyali gözle doğrulamak için harikadır. Bir chirp'te köşegen çizgi, saf tonda tek dikey sütun, gürültüde homojen bir bulut görürsünüz. Renkli çıktı için ANSI kaçış kodları (`\033[38;5;...m`) ekleyip gerçek bir şelale (waterfall) elde edebilirsiniz.

32. Proje 4 — Enstrüman Akort Cihazı (Pitch Detection)

Bir gitar veya sesin temel frekansını (pitch) bulup en yakın nota ile kaç sent (cent) saptığını gösteren bir akort cihazı. Frekansı, FFT yerine **otokorelasyonla** buluruz — gürültülü ve harmonik zengini seslerde daha güvenilirdir.

```

1  #include <vector>
2  #include <cmath>
3  #include <cstdio>
4  #include <string>
5
6  // Otokorelasyonla temel frekansı kestir. 0 = perde bulunamadi.
7  double detectPitch(const std::vector<float>& x, double fs,
8                    double minHz = 50.0, double maxHz = 1000.0) {
9      const size_t minLag = static_cast<size_t>(fs / maxHz);
10     const size_t maxLag = static_cast<size_t>(fs / minHz);
11     double bestCorr = 0; size_t bestLag = 0;
12
13     // Her olası periyot (lag) için otokorelasyonu hesapla
14     for (size_t lag = minLag; lag <= maxLag && lag < x.size(); ++lag) {
15         double corr = 0;
16         for (size_t i = 0; i + lag < x.size(); ++i) corr += x[i] * x[i + lag];
17         if (corr > bestCorr) { bestCorr = corr; bestLag = lag; }
18     }
19     return bestLag ? fs / bestLag : 0.0;
20 }
21
22 // Frekanstan en yakın nota adı + sent sapması.
23 std::string nearestNote(double freq, double* centsOut) {
24     static const char* names[] = {"C", "C#", "D", "D#", "E", "F",
25                                   "F#", "G", "G#", "A", "A#", "B"};
26     // A4 = 440 Hz referans; yarı-ton = 2^(1/12)
27     double midi = 69.0 + 12.0 * std::log2(freq / 440.0);
28     int nearest = static_cast<int>(std::lround(midi));
29     *centsOut = (midi - nearest) * 100.0; // sent cinsinden sapma
30     int octave = nearest / 12 - 1;
31     return std::string(names[(nearest % 12 + 12) % 12]) + std::to_string(octave);
32 }
33
34 int main() {
35     const double fs = 44100.0;
36     // Test: hafif akortsuz bir "La" (441.5 Hz) üret
37     std::vector<float> x;
38     for (int n = 0; n < 8192; ++n)
39         x.push_back(0.7f * std::sin(2*M_PI*441.5*n/fs)
40                   + 0.2f * std::sin(2*M_PI*883.0*n/fs)); // + 2. harmonik
41
42     double f = detectPitch(x, fs);
43     double cents;
44     std::string note = nearestNote(f, &cents);

```

```

45     printf("Frekans: %.2f Hz -> Nota: %s (%+.1f sent)\n",
46           f, note.c_str(), cents);    // "A4 (+5.9 sent)" -> hafif tiz
47     return 0;
48 }

```

Neden otokorelasyon?

FFT tepesi temel frekansı verir ama harmonikler güçlüyse (gitar, insan sesi) yanlış oktava atlayabilir. Otokorelasyon, işaretin *periyodunu* doğrudan arar ve harmoniklere karşı daha gürbüzdür. Daha da iyisi **YIN** ve **pYIN** algoritmalarıdır (fark fonksiyonu tabanlı); profesyonel akort uygulamaları bunları kullanır.

33. Projeleri Derleme (CMake)

Dört projeyi de derleyen tek bir CMake dosyası. Her proje bu belgedeki yardımcılarını (writeWav, fft, hannWindow, goertzel, designLowpass...) tek bir dsp.hpp başlığında toplayıp #include ettiğinizi varsayar.

```

1  cmake_minimum_required(VERSION 3.15)
2  project(dsp_projeler LANGUAGES CXX)
3
4  set(CMAKE_CXX_STANDARD 17)
5  set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7  # Optimizasyon: DSP kodu SIMD'den çok faydalanır
8  if(NOT MSVC)
9      add_compile_options(-O3 -march=native)
10 endif()
11
12 # Her proje bagimsiz bir calistirilabilir
13 add_executable(dtmf      proje1_dtmf.cpp)
14 add_executable(fsk_modem proje2_modem.cpp)
15 add_executable(spektrum  proje3_spektrum.cpp)
16 add_executable(akort     proje4_akort.cpp)
17
18 # Ortak baslik dizini (dsp.hpp burada)
19 foreach(t dtmf fsk_modem spektrum akort)
20     target_include_directories(${t} PRIVATE ${CMAKE_SOURCE_DIR}/include)
21 endforeach()

```

```

1  # Derle ve calistir
2  mkdir build && cd build
3  cmake .. -DCMAKE_BUILD_TYPE=Release
4  cmake --build .
5
6  ./dtmf                # -> Cozulen tus: 5
7  ./fsk_modem           # -> modem.wav uretir, metni geri cozer
8  ./spektrum            # -> terminalde kosegen chirp spektrumu
9  ./akort               # -> A4 (+5.9 sent)
10
11 # Uretilen sesleri dinle / spektrogramla:
12 sox modem.wav -n spectrogram -o modem.png

```

İpucu

Bu dört projeyi genişletmek için doğal fikirler: DTMF çözücüye gerçek .wav girişi ve gürültü reddi; FSK modeme Viterbi FEC ve gerçek ses kartı G/Ç (PortAudio/RtAudio); spektrum analizörüne ncurses ile canlı kaydırılan şelale; akort cihazına mikrofon girişi ve grafik iğne göstergesi. Her biri, bu belgedeki yapı taşlarının üstüne birkaç yüz satırla oturur.

KISIM 9

Matematiksel Temeller

Bu kısım, belge boyunca kod hâline getirdiğimiz işlemlerin altındaki matematiği adım adım türetir. Amaç, her algoritmanın *neden* çalıştığını sezgiyle değil, ispatla görmektir. Her sonucu bir önceki koda geri bağlayacağız.

34. Karmaşık Üsteller ve Diklik (Orthogonality)

Tüm Fourier teorisi tek bir yapı taşına dayanır: karmaşık üstel $e^{j\omega t}$. Euler formülü $e^{j\theta} = \cos \theta + j \sin \theta$ 'dan iki temel kimlik çıkar:

$$\cos \theta = \frac{e^{j\theta} + e^{-j\theta}}{2}, \quad \sin \theta = \frac{e^{j\theta} - e^{-j\theta}}{2j}$$

Bu, “bir kosinüs, biri $+\omega$ diğeri $-\omega$ 'da dönen iki fazörün toplamıdır” demenin cebirsel hâlidir — gerçek sinüslerin neden hem pozitif hem negatif frekans içerdiğini (Kısım 4) buradan görürüz.

34.1 Diklik İlişkisi

Fourier'in işleyebilmesinin sebebi, farklı frekanstaki üstellerin **dik** (birbirine “bağımsız”) olmasıdır. Sürekli durumda bir T periyodu üzerinde:

$$\frac{1}{T} \int_0^T e^{j2\pi mt/T} e^{-j2\pi nt/T} dt = \begin{cases} 1, & m = n \\ 0, & m \neq n \end{cases}$$

Ayrık durumda (N örnek üzerinde) tam karşılığı:

$$\sum_{n=0}^{N-1} e^{j2\pi kn/N} e^{-j2\pi ln/N} = \begin{cases} N, & k = l \pmod{N} \\ 0, & \text{diğer} \end{cases}$$

İkincisi bir **geometrik seri toplamıdır**: $k \neq l$ için ortak çarpan $r = e^{j2\pi(k-l)/N} \neq 1$ 'dir ve $\sum_{n=0}^{N-1} r^n = \frac{r^N - 1}{r - 1} = 0$ (çünkü $r^N = e^{j2\pi(k-l)} = 1$). İşte DFT'nin *tersinin* neden var olduğunu kanıtı budur — her frekans bileşenini diğerlerine karışmadan geri çekebiliriz.

35. Fourier Zinciri: Seriden DFT'ye

Dört Fourier gösterimi vardır; hangisini kullanacağımız işaretin zaman ve frekansta sürekli mi kesikli mi olduğuna bağlıdır:

Dönüşüm	Zaman	Frekans
Fourier Serisi (FS)	sürekli, periyodik	kesikli, sonsuz
Fourier Dönüşümü (FT)	sürekli, aperiodyk	sürekli, sonsuz
Ayrık-Zaman FT (DTFT)	kesikli, aperiodyk	sürekli, periyodik
Ayrık Fourier Dön. (DFT)	kesikli, periyodik	kesikli, periyodik

35.1 Fourier Serisi ve Dönüşümü

Periyodu T olan bir işaret, harmoniklerin toplamıdır:

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{j2\pi kt/T}, \quad c_k = \frac{1}{T} \int_0^T x(t) e^{-j2\pi kt/T} dt$$

Katsayı formülü doğrudan yukarıdaki diklikten çıkar: her iki tarafı $e^{-j2\pi lt/T}$ ile çarpıp integre edin, dik olmayan tek terim $k = l$ hayatta kalır. Periyodu sonsuza götürürsek ($T \rightarrow \infty$) toplam integrale, kesikli k/T frekansları sürekli f 'e dönüştür ve **Fourier dönüşümünü** elde ederiz:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt, \quad x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df$$

35.2 DTFT'den DFT'ye: Kodun Türetimi

İşareti örnekleyip ($t = nT_s$) kesikli yaparsak DTFT'yi elde ederiz:

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

Burada $\omega = 2\pi f/f_s$ (radyan/örnek, Kısım 1'deki w). Bu hâlâ ω 'da sürekli — bilgisayarda saklanamaz. Son adım, frekans eksenini de N noktada örnekleme: $\omega_k = 2\pi k/N$. Bu bize tam olarak Kısım 2'de kodladığımız **DFT**'yi verir:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

Yani `dft()` fonksiyonu, sürekli Fourier dönüşümünün hem zamanda hem frekansta örneklenmiş, tam hesaplanabilir hâlidir. Diklik ilişkisi de ters dönüşümü (`idft()`) ve $1/N$ çarpanını doğrular.

35.3 FFT'nin Böl-ve-Fethet İspatı

FFT'nin $O(N \log N)$ 'i, DFT toplamını çift ve tek indekslere ayırmaktan doğar. $W_N = e^{-j2\pi/N}$ yazarsak:

$$X[k] = \sum_{n \text{ çift}} x[n] W_N^{kn} + \sum_{n \text{ tek}} x[n] W_N^{kn} = \underbrace{\sum_{m=0}^{N/2-1} x[2m] W_{N/2}^{km}}_{E[k]} + W_N^k \underbrace{\sum_{m=0}^{N/2-1} x[2m+1] W_{N/2}^{km}}_{O[k]}$$

çünkü $W_N^{2km} = W_{N/2}^{km}$. Ayrıca $W_N^{k+N/2} = -W_N^k$ (simetri) olduğundan:

$$X[k] = E[k] + W_N^k O[k], \quad X[k + N/2] = E[k] - W_N^k O[k]$$

İşte `fftRecursive` içindeki *kelebek* tam olarak budur: iki yarı-boy DFT hesapla, twiddle çarpanı W_N^k ile birleştir. Her katmanda iş yarıya inerken $\log_2 N$ katman olması $O(N \log N)$ 'i verir.

36. Örneklemeye Teoreminin Türetimi

Nyquist teoremini (Kısım 2) artık ispatlayabiliriz. Örneklemeye, işareti bir **darbe dizisiyle** (Dirac tarağı) çarpmaaktır:

$$x_s(t) = x(t) \cdot \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$$

Dirac tarağının Fourier dönüşümü yine bir Dirac tarağıdır (aralık $f_s = 1/T_s$). **Çarpımın dönüşümü konvolüsyondur** (bir sonraki bölüm) olduğundan, örneklenmiş işaretin spektrumu, orijinal spektrumun f_s aralıklarıyla *sonsuz kez kopyalanmış* hâlidir:

$$X_s(f) = f_s \sum_{k=-\infty}^{\infty} X(f - kf_s)$$

İşte aliasing'in kaynağı budur. Eğer $X(f)$ 'in bant genişliği $B < f_s/2$ ise kopyalar üst üste binmez ve orijinali bir alçak geçiren filtreyle kusursuz geri alabiliriz. Ama $B > f_s/2$ ise komşu kopyalar örtüşür — yüksek frekanslar düşüklerin üstüne katlanır (Kısım 2'deki `aliasedFrequency`). Kritik sınır $f_s = 2B$ 'dir; **Nyquist oranı** budur.

İdeal geri çatım

Örtüşme yoksa, orijinal işaret örneklerden şu interpolasyonla *tam* olarak geri gelir: $x(t) = \sum_n x[n] \text{sinc}((t - nT_s)/T_s)$. Bu, kesikli örneklerin neden sürekli işareti eksiksiz temsil edebildiğinin matematiksel güvencesidir — sinc çekirdeği ideal alçak geçiren filtrenin dürtü yanıtıdır, ki Kısım 3'te FIR tasarımında tam bunu kullandık.

37. Konvolüsyon ve Modülasyon Teoremleri

Filtreleme ve modülasyonun tüm matematiği iki eşleştirmeye dayanır. Bunlar Fourier dönüşümünün en güçlü özellikleridir.

37.1 Konvolüsyon Teoremi

Zamanda konvolüsyon, frekansta çarpmadır (ve tersi):

$$y(t) = (x * h)(t) \iff Y(f) = X(f) H(f)$$

İspat, konvolüsyon integralinin dönüşümünü alıp değişken değiştirmektir; $e^{-j2\pi f\tau}$ çarpanı ayırır. Bu teoremin iki büyük sonucu var:

- **Filtrelemenin anlamı:** Kısım 3'teki FIR/IIR bir konvolüsyondur; frekans alanında bu, işaretin spektrumunu filtrenin *frekans yanıtı* $H(f)$ ile çarpmaaktır. “Alçak geçiren” demek, $H(f)$ 'in düşük frekanslarda ≈ 1 , yüksekte ≈ 0 olması demektir.
- **Hızlı konvolüsyon:** $O(N^2)$ 'lik konvolüsyon yerine $\text{IFFT}\{\text{FFT}\{x\} \cdot \text{FFT}\{h\}\}$ ile $O(N \log N)$ 'de sonuç — uzun FIR'lar için overlap-add yönteminin temeli.

37.2 Modülasyon (Frekans Kaydırma) Teoremi

Konvolüsyon teoreminin ikizi: zamanda bir üstelle çarpma, frekansta kaydırmadır:

$$x(t) e^{j2\pi f_0 t} \iff X(f - f_0)$$

İşte Kısım 4'teki `Nco/frequencyShift`'in ispatı: bir IQ akışını $e^{-j2\pi f_0 n/f_s}$ ile çarpmak, spektrumunu $-f_0$ kadar ötelere ve istediğimiz kanalı DC'ye getirir. Gerçek bir kosinüsle çarpsaydık ($\cos = \frac{1}{2}(e^j + e^{-j})$) spektrum $hem +f_0 hem -f_0$ 'a kopyalanır — bu istenmeyen “görüntü” (image) tam olarak IQ'nun (tek taraflı spektrum) neden gerekli olduğunu gösterir.

37.3 Parseval Teoremi

Enerji, zaman ve frekans alanında korunur:

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2$$

Bu yüzden bir işaretin gücünü spektrumdan ($|X[k]|^2$, yani PSD) hesaplayabiliriz — Kısım 2'deki `powerSpectrumDb` ve pencere kazancı telafisi bu eşitliğe dayanır.

38. Analitik İşaret ve Anlık Frekans

Son olarak, tüm aç-demodulatorlarının (FM/PM) altındaki matematiği kesinleştirelim.

38.1 Anlık Frekansın Tanımı

Karmaşık bir işareti kutupsal yazalım: $z(t) = a(t)e^{j\phi(t)}$. Buradaki $a(t) = |z(t)|$ **anlık zarf**, $\phi(t) = \angle z(t)$ **anlık fazdır**. **Anlık frekans**, fazın türevidir:

$$f_i(t) = \frac{1}{2\pi} \frac{d\phi(t)}{dt}$$

Bu tanım FM demodülasyonunu doğrudan açıklar: FM'de $\phi(t) = 2\pi k_f \int m$, dolayısıyla $\frac{1}{2\pi} \phi'(t) = k_f m(t)$ — yani fazın türevi tam olarak mesajdır (bir ölçek dışında).

38.2 Polar Diskriminatörün Türetimi

Ayrık zamanda türev, ardışık fazların farkıdır: $\phi[n] - \phi[n-1]$. Bunu `atan2` sarmalanması olmadan hesaplamamanın yolu, Kısım 5'teki eşlenik çarpımıdır. $z[n] = a[n]e^{j\phi[n]}$ için:

$$z[n] z^*[n-1] = a[n] a[n-1] e^{j(\phi[n] - \phi[n-1])}$$

Genlikler pozitif reel olduğundan bu çarpımın *açısı* tam olarak faz farkıdır:

$$\angle(z[n] z^*[n-1]) = \phi[n] - \phi[n-1] \approx 2\pi f_i T_s$$

İşte `FmDemod::process`'in matematiksel ispatı budur. `std::arg` sonucu doğal olarak $[-\pi, \pi]$ aralığında verdiği için, faz $\pm\pi$ 'yi geçse bile sonuç sarmalanmadan doğru çıkar — bu yüzden ayrı bir “unwrap” adımına gerek kalmaz.

38.3 Hilbert Dönüşümü ve Analitik İşaretin Türetimi

Gerçek bir $x(t)$ 'nin analitik karşılığı $x_a(t) = x(t) + j\hat{x}(t)$ 'dir; burada $\hat{x}$ **Hilbert dönüşümüdür**. Amaç, negatif frekansları yok edip tek taraflı bir spektrum elde etmektir. Frekans alanında Hilbert filtresi $H(f) = -j \operatorname{sgn}(f)$ 'dir; bu, tüm pozitif frekansları -90° , negatifleri $+90^\circ$ döndürür. Bunun sonucunda:

$$X_a(f) = X(f)(1 + \operatorname{sgn}(f)) = \begin{cases} 2X(f), & f > 0 \\ X(0), & f = 0 \\ 0, & f < 0 \end{cases}$$

İşte Kısım 4'teki `analyticSignal` fonksiyonunun “pozitif frekansları 2 ile çarp, negatifleri sıfırla” adınının kaynağı budur. Böylece $|x_a|$ zarfı (AM demod) ve $\angle x_a$ fazı (FM/PM demod) doğrudan okunabilir hâle gelir — tüm demodulatorları birkaç satıra indiren şey bu matematiktir.

İpucu

Bütün belgeyi tek cümlede özetlersek: *bir işareti karmaşık bant temeline taşıdığınızda*, genlik `std::abs`, faz `std::arg`, frekans ise ardışık fazların farkıdır. Modülasyon bu üçünden birine bilgi gömer; demodülasyon onu geri okur. Geri kalan her şey — örnekleme, FFT, filtreler, mikser — bu okumayı temiz ve verimli yapmak içindir.